Laporan M2 Tugas Besar 1
IF2010 Pemrograman Berorientasi Objek 2025/2026
Nimonspoli

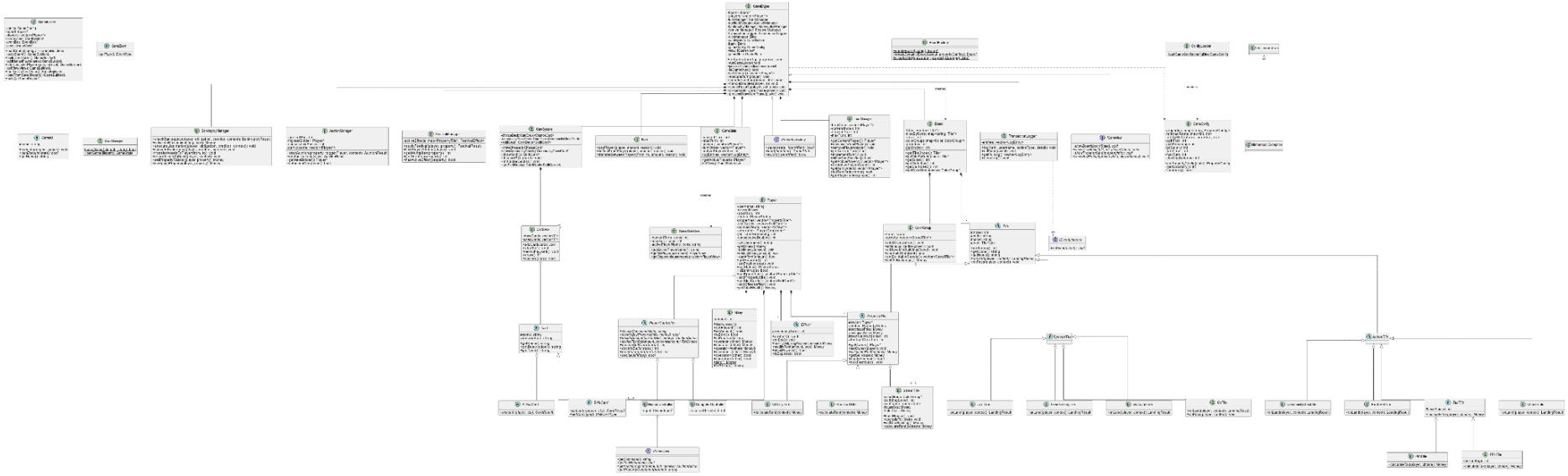
Kode Kelompok : BCC
Nama Kelompok : Burnt Cheesecake

1. 13524026 Made Branenda Jordhy
2. 13524028 Muhammad Nur Majiid
3. 13524034 Jason Edward Salim
4. 13524067 Bryan Pratama Putra Hendra
5. 13524068 Athilla Zaidan Zidna Fann

BCC - Burnt Cheese Cake

1. Diagram Kelas

Dikarenakan Diagram kelas yang terlalu kompleks dan tidak dapat ditampilkan dengan jelas pada dokumen ini, diagram kelas secara keseluruhan dapat diakses melalui link berikut: DrawIO atau dapat diakses juga melalui link berikut: Drive



Gambar 1.1 Diagram Kelas Keseluruhan (dengan Relasi tiap kelasnya)

2. Deskripsi Kelas

2.1. Kelas GameEngine

GameEngine merupakan orkestrator utama permainan. Memiliki semua manajer dan menjalankan game loop dari awal hingga kondisi akhir terpenuhi. Satu-satunya kelas yang boleh memodifikasi GameState.

2.1.1. Atribut Kelas

Atribut	Tipe	Keterangan
board	Board*	Papan permainan yang berisi semua tile.
players	vector<Player*>	Daftar semua pemain aktif dan tidak aktif.
turnManager	TurnManager	Mengelola urutan dan penghitungan giliran.
auctionManager	AuctionManager	Menangani seluruh proses lelang.
bankruptcyManager	BankruptcyManager	Menangani likuidasi dan kebangkrutan.
festivalManager	FestivalManager	Melacak efek festival aktif.
transactionLogger	TransactionLogger	Mencatat semua aksi dan transaksi.
diceManager	Dice	Melempar dan menyimpan hasil dadu.
cardSystem	CardSystem	Mengelola tiga deck kartu.
bank	Bank	Sumber dan penampung uang tak terbatas.
gameConfig	GameConfig	Semua nilai konfigurasi permainan.

view	IGameView*	Antarmuka tampilan; tidak pernah cout langsung.
gameState	GameState	Snapshot state penuh untuk save/load.

2.1.2. Method Kelas

Method	Return	Keterangan
initialize(config, players)	void	Membangun board, deck, dan objek pemain dari konfigurasi.
runGameLoop()	void	Loop utama: berulang hingga isGameOver() true.
processCommand(command)	void	Menerima string perintah dan mendelegasikan ke CommandRegistry.
isGameOver()	bool	True jika ronde maks tercapai atau hanya 1 pemain tersisa.
getWinner()	vector<Player*>	Mengembalikan pemain dengan kekayaan tertinggi.
executeTurn(player)	void	Mengelola satu giliran penuh, dadu -> pindah -> landing.
handleLanding(player, tile)	void	Memproses LandingResult dari Tile::onLand().
handleDoubles(player, n)	void	Menangani dadu kembar, 3x berturut = masuk penjara.
checkPassGo(player, old, new)	void	Memberi gaji jika melewati petak GO.
processSkillCardPhase(player)	void	Menawarkan kesempatan penggunaan kartu skill.
processDrawCardPhase(player)	void	Mengambil kartu Chance atau Community Chest.

2.1.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Board, TurnManager, Bank	Owns (komposisi) semua manajer dan komponen inti.
IGameView	Dependency, memanggil view untuk setiap output tampilan.
GameState	Owns, memperbarui snapshot setiap akhir aksi penting.
ConfigLoader	Dependency tidak langsung, menerima GameConfig hasil parsing.

2.2. Kelas GameBuilder

Kelas GameBuilder bertugas untuk membangun dan merakit seluruh sistem permainan secara bertahap menggunakan pola Builder. Kelas ini juga memisahkan proses konstruksi yang kompleks dari logika permainan sehingga main.cpp cukup memanggil rantai metode yang mudah dibaca dan tidak mudah salah urutan.

2.2.1. Atribut Kelas

Atribut	Tipe	Keterangan
config	GameConfig	Konfigurasi lengkap yang dimuat dari file; diisi oleh loadConfig().
board	Board*	Papan yang dibangun oleh buildBoard(); awalnya nullptr.
players	vector<Player*>	Daftar pemain yang ditambahkan secara bertahap; awalnya kosong.
cardSystem	CardSystem*	Sistem deck kartu yang dibangun oleh buildCardDecks(); awalnya nullptr.

eventBus	EventBus*	Bus event yang menghubungkan GameEngine dengan semua subscriber UI.
view	IGameView*	Antarmuka tampilan yang akan diinjeksi ke GameEngine; diset oleh setView().

2.2.2. Method Kelas

Method	Return	Keterangan
loadConfig(configDir)	GameBuilder&	Panggil ConfigLoader untuk mengisi atribut config. Wajib dipanggil pertama.
buildBoard()	GameBuilder&	Panggil BoardFactory::createBoard(config) dan simpan hasilnya ke board.
buildCardDecks()	GameBuilder&	Inisialisasi CardSystem dan kocok semua deck menggunakan data konfigurasi.
addHumanPlayer(name)	GameBuilder&	Buat Player baru dengan HumanController; tambahkan ke daftar players.
addComputerPlayer(name, aiLevel)	GameBuilder&	Buat Player baru dengan ComputerController sesuai aiLevel; tambahkan ke players.
setView(view)	GameBuilder&	Simpan pointer IGameView* yang akan diinjeksi ke GameEngine saat build().
randomizeTurnOrder()	GameBuilder&	Acak urutan dalam vector players untuk menentukan siapa giliran pertama.
loadFromSave(filepath)	GameBuilder&	Panggil SaveManager::loadGame() dan pulihkan seluruh state ke atribut builder.

build()	GameEngine*	Rakit semua komponen menjadi GameEngine; lempar exception jika ada yang belum diset.

2.2.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
ConfigLoader	Dipanggil dalam loadConfig() untuk mengisi GameConfig dari file.
BoardFactory	Dipanggil dalam buildBoard() untuk menghasilkan Board* beserta semua tile-nya.
CardSystem	Diinstansiasi dalam buildCardDecks() dan diserahkan ke GameEngine.
HumanController	Diinstansiasi dalam addHumanPlayer() dan diinjeksi ke Player.
ComputerController	Diinstansiasi dalam addComputerPlayer() dan diinjeksi ke Player.
SaveManager	Dipanggil dalam loadFromSave() untuk memulihkan state permainan tersimpan.
GameEngine	Produk akhir build(); menerima semua komponen yang telah dirakit.
IGameView	Diterima via setView() dan diinjeksi ke GameEngine saat build().
EventBus	Dibuat oleh builder dan diserahkan ke GameEngine beserta semua subscriber.

2.3. Kelas Bank

Kelas Bank bertugas sebagai entitas keuangan sentral dengan sumber uang tak terbatas. Menjadi satu-satunya titik masuk untuk semua transfer uang agar konsistensi saldo terjaga.

2.3.1. Atribut Kelas

Atribut	Tipe	Keterangan
-	-	Bank tidak memiliki atribut state; ia murni menyediakan metode transaksi.

2.3.2. Method Kelas

Method	Return	Keterangan
payPlayer(player, amount, reason)	void	Setor uang dari bank ke saldo pemain.
collectFromPlayer(player, amount, reason)	void	Tarik uang dari saldo pemain ke bank.
transferBetweenPlayers(from, to, amount, reason)	void	Pindahkan uang antar pemain secara atomik.

2.3.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Player	Memodifikasi money via addMoney() / deductMoney().
Money	Menggunakan operator += dan -= milik Money untuk aritmatika.
GameEngine	Dipanggil untuk setiap transaksi di handleLanding().
AuctionManager	Dipanggil untuk memproses pembayaran pemenang lelang.
BankruptcyManager	Dipanggil saat aset pemain dialihkan atau dikembalikan ke bank.

TransactionLogger	Menerima parameter reason untuk dicatat di log.

2.4. Kelas TurnManager

Kelas TurnManager mengelola urutan giliran dan penghitungan ronde. Memastikan rotasi berjalan benar meskipun ada pemain yang keluar karena bangkrut.

2.4.1. Atribut Kelas

Atribut	Tipe	Keterangan
turnOrder	vector<Player*>	Urutan giliran pemain yang masih aktif.
currentIndex	int	Indeks pemain yang sedang giliran.
currentTurn	int	Nomor ronde saat ini; bertambah setelah satu siklus penuh.
maxTurn	int	Batas ronde maksimal dari GameConfig.

2.4.2. Method Kelas

Method	Return	Keterangan
getCurrentPlayer()	Player*	Mengembalikan pemain yang giliran sekarang.
advanceToNextPlayer()	void	Maju ke pemain berikutnya, lewati yang BANKRUPT.

removePlayer(player)	void	Hapus pemain dari urutan saat dinyatakan bangkrut.
getCurrentTurn()	int	Mengembalikan nomor ronde saat ini.
incrementTurn()	void	Tambah counter ronde setelah satu siklus penuh.
isMaxTurnReached()	bool	True jika currentTurn >= maxTurn.
getActivePlayers()	vector<Player*>	Semua pemain berstatus ACTIVE.
getActivePlayerCount()	int	Jumlah pemain yang masih aktif.
getTurnOrder()	vector<Player*>	Mengembalikan seluruh urutan giliran.
setTurnOrder(order)	void	Mengatur ulang urutan (dipakai setelah load save).
getPlayerIndex(player)	int	Indeks suatu pemain dalam urutan giliran.

2.4.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameEngine	Dipanggil setiap akhir giliran untuk mendapatkan pemain berikutnya.
Player	Membaca PlayerStatus untuk memfilter pemain aktif.
BankruptcyManager	Memanggil removePlayer() saat pemain dinyatakan bangkrut.

2.5. Kelas AuctionManager

Mengelola proses lelang properti dari awal hingga penentuan pemenang. Lelang dipicu saat pemain menolak membeli properti atau properti dikembalikan ke bank akibat kebangkrutan.

2.5.1. Atribut Kelas

Atribut	Tipe	Keterangan
currentBid	int	Nilai penawaran tertinggi saat ini.
highestBidder	Player*	Pemain yang saat ini memegang penawaran tertinggi.
consecutivePasses	int	Jumlah pemain berturut-turut yang melewati giliran.
participants	vector<Player*>	Daftar pemain yang ikut serta dalam lelang.

2.5.2. Method Kelas

Method	Return	Keterangan
startAuction(property, players, triggerPlayer, context)	AuctionResult	Titik masuk utama; inisialisasi lelang dan panggil runAuctionLoop().
runAuctionLoop(view)	AuctionResult	Loop penawaran hingga kondisi selesai terpenuhi.
getNextBidder()	Player*	Ambil peserta berikutnya dalam rotasi.
isAuctionOver(count)	bool	True jika semua peserta aktif sudah pass.

2.5.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameEngine	Dipanggil saat pemain menolak beli properti.
BankruptcyManager	Dipanggil saat properti dikembalikan ke bank.
PlayerController	Memanggil decideAuction() untuk setiap keputusan peserta.
IGameView	Menampilkan status lelang terkini via showAuctionState().
Bank	Memproses pembayaran harga akhir lelang.

2.6. Kelas BankruptcyManager

Kelas BankruptcyManager menangani proses likuidasi dan kebangkrutan. Urutan yang diterapkan adalah menghitung kemampuan likuidasi lalu meminta pemain menjual aset dan terakhir menyatakan bangkrut jika uang tetap tidak cukup.

2.6.1. Atribut Kelas

Atribut	Tipe	Keterangan
-	-	Tidak memiliki atribut state. Semua data diambil dari parameter context.

2.6.2. Method Kelas

Method	Return	Keterangan
checkBankruptcy(player, obligation, creditor, context)	BankruptcyResult	Titik masuk, periksa apakah saldo + likuidasi cukup.
calculateMaxLiquidation(player)	Money	Total uang yang bisa diperoleh dari penjualan/gadai semua aset.
executeLiquidation(player, obligation, creditor, context)	void	Panduan interaktif menjual/menggadaikan aset hingga cukup.
executeBankruptcy(player, creditor, context)	void	Pindahkan aset ke kreditur atau bank; ubah status jadi BANKRUPT.
transferAssetsToPlayer(from, to)	void	Pindahkan semua properti dari pemain bangkrut ke kreditur.
returnAssetsToBank(player, context)	void	Kembalikan properti ke bank dan picu lelang tiap properti.
sellPropertyToBank(player, property)	Money	Jual satu properti ke bank pada nilai jual.
mortgageProperty(player, property)	Money	Gadaikan satu properti untuk mendapat setengah harga beli.

2.6.3. FRelasi Kelas

Kelas Terkait	Bentuk Relasi
GameEngine	Dipanggil setiap kali pemain tidak mampu membayar kewajiban.
AuctionManager	Dipanggil untuk melelang properti yang dikembalikan ke bank.

TurnManager	Memanggil removePlayer() setelah kebangkrutan dieksekusi.
PlayerController	Memanggil decideLiquidation() dalam proses likuidasi interaktif.
TransactionLogger	Mencatat setiap langkah penjualan dan pengalihan aset.

2.7. Kelas FestivalManager

Kelas FestivalManager mengelola efek pengganda sewa yang diaktifkan pemain melalui petak Festival. Kelas ini juga melacak durasi dan nilai pengganda aktif per properti.

2.7.1. Atribut Kelas

Atribut	Tipe	Keterangan
activeEffects	map<PropertyTile*, FestivalEffect>	Peta efek festival aktif; pencarian O(1) per properti.

2.7.2. Method Kelas

Method	Return	Keterangan
applyFestival(player, property)	FestivalResult	Aktifkan atau gandakan efek festival pada properti pilihan.
tickPlayerEffects(player)	void	Kurangi durasi semua efek aktif; hapus yang sudah habis.
getMultiplier(property)	int	Kembalikan pengganda sewa aktif saat ini (1, 2, 4, atau 8).

getDuration(property)	int	Kembalikan sisa durasi efek dalam satuan giliran.
hasActiveEffect(property)	bool	True jika properti tersebut sedang memiliki efek festival.

2.7.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
FestivalTile	Memanggil applyFestival() saat pemain mendarat dan memilih properti.
StreetTile	Memanggil getMultiplier() dalam calculateRent() untuk sewa final.
GameEngine	Memanggil tickPlayerEffects() setiap akhir giliran pemain.
FestivalEffect	Menyimpan data efek (pengganda, durasi, pemilik) dalam struct ini.

2.8. Kelas CardSystem

Kelas CardSystem menjadi fasad untuk tiga deck kartu. Kelas ini menyembunyikan detail manajemen deck dari GameEngine dan memastikan kartu yang habis dikocok ulang secara otomatis.

2.8.1. Atribut Kelas

Atribut	**Tipe**	**Keterangan**
chanceDeck	CardDeck<ChanceCard>	Deck 3 kartu Chance.

communityChestDeck	CardDeck<CommunityChestCard>	Deck 3 kartu Community Chest.
skillDeck	CardDeck<SkillCard>	Deck 6 jenis kartu Skill.

2.8.2. Method Kelas

Method	Return	Keterangan
drawChance()	ChanceCard*	Ambil kartu dari chanceDeck; kocok ulang jika kosong.
drawCommunityChest()	CommunityChestCard*	Ambil kartu dari communityChestDeck; kocok ulang jika kosong.
drawSkill()	SkillCard*	Ambil kartu skill; diserahkan ke tangan pemain.
discardSkill(card)	void	Kembalikan kartu skill yang digunakan ke discard pile.
initializeDecks()	void	Buat semua kartu dan kocok saat permainan dimulai.
getSkillDeck()	CardDeck<SkillCard>&	Akses deck skill untuk keperluan serialisasi save/load.

2.8.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameEngine	Memanggil draw* saat pemain mendarat di petak kartu.
CardDeck<T>	Menggunakan template class ini untuk ketiga deck.

ChanceCard, CommunityChestCard, SkillCard	Hierarki kartu yang diinstansiasi dalam initializeDecks().
SaveManager	Mengakses getSkillDeck() untuk serialisasi urutan kartu.

2.9. Kelas Money

Value class yang membungkus nilai uang sebagai objek, bukan sebagai primitive `int` biasa. Kelas ini menyediakan operator overloading untuk seluruh operasi keuangan dalam permainan (penambahan, pengurangan, perbandingan), sehingga setiap transaksi uang antar pemain, bank, dan properti dilakukan secara aman dan ekspresif.

2.9.1. Atribut Kelas

Atribut	Tipe	Keterangan
amount	int	Nilai nominal uang dalam satuan M (Monopoly money). Disimpan sebagai integer untuk menghindari error pembulatan floating-point.

2.9.2. Method Kelas

Method	Return	Keterangan
Money(amount)	constructor	Membuat objek Money dengan nilai awal tertentu. Default 0 jika tidak diisi.
getAmount()	int	Return nilai nominal uang sebagai integer

setAmount()	void	Mengubah nilai nominal uang. Digunakan oleh sistem load game.
isZero()	bool	True jika nilai uang sama dengan 0.
isPositive()	bool	True jika nilai uang lebih dari 0.
operator+(other)	Money	Menjumlahkan dua objek Money, mengembalikan Money baru.
operator-(other)	Money	Mengurangi Money dengan Money lain, mengembalikan Money baru.
operator+=(other)	Money&	Menambahkan Money lain ke instance ini (in-place). Digunakan saat pemain menerima gaji, sewa, atau hadiah.
operator-=(other)	Money&	Mengurangi instance ini dengan Money lain (in-place). Digunakan saat pemain membayar sewa, pajak, atau denda.
operator<(other)	bool	True jika nilai instance ini kurang dari Money lain. Dipakai untuk validasi "uang cukup atau tidak".
operator>(other)	bool	True jika nilai instance ini lebih dari Money lain.
operator<=(other)	bool	True jika nilai instance ini kurang dari atau sama dengan Money lain.
operator>=(other)	bool	True jika nilai instance ini lebih dari atau sama dengan Money lain.
operator==(other)	bool	True jika dua Money memiliki nilai nominal yang sama.
operator!=(other)	bool	True jika dua Money memiliki nilai nominal berbeda.
static zero()	Money	Factory method yang mengembalikan objek Money bernilai 0.

toString()	string	Mengembalikan representasi teks uang dalam format "M{nilai}" (misal "M1500").

2.9.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Player	Player memiliki atribut money: Money sebagai saldo tunai. Seluruh transaksi pemain melewati operator Money.
Bank	Bank menggunakan Money saat mentransfer dana antar pemain via payPlayer(), collectFromPlayer(), dan transferBetweenPlayers().
PropertyTile	Setiap properti menyimpan purchasePrice: Money dan mortgageValue: Money yang dibandingkan terhadap saldo pemain.
AuctionManager	Validasi bid menggunakan operator < dan > untuk memastikan bid lebih tinggi dari currentBid dan tidak melebihi saldo pemain.
BankruptcyManager	Membandingkan obligation: Money dengan total potensi likuidasi pemain menggunakan operator perbandingan.
TaxTile	Pajak flat dan persentase menghasilkan objek Money yang langsung dikurangkan dari saldo pemain.
MoneyChangedEvent	Event ini membawa data oldAmount: Money dan newAmount: Money untuk dipublikasikan ke EventBus.

2.10. Kelas Player

Merepresentasikan satu pemain dalam permainan Nimonspoli. Player adalah entitas inti yang memiliki saldo uang, posisi di papan, daftar properti, kartu kemampuan di tangan, dan efek aktif yang sedang berlaku. Player tidak mengambil keputusan sendiri, semua keputusan

didelegasikan ke PlayerController (Strategy Pattern), sehingga Player yang sama bisa dikendalikan oleh manusia maupun AI tanpa perubahan internal.

2.10.1. Atribut Kelas

Atribut	Tipe	Keterangan
username	int	Nama unik pemain yang ditampilkan di UI dan log transaksi.
money	Money	Saldo uang tunai pemain saat ini.
position	int	Indeks petak tempat pemain berada di papan (0-based).
status	PlayerStatus	Status pemain: ACTIVE, BANKRUPT, atau JAILED.
properties	vector<PropertyTile*>	Daftar pointer ke seluruh properti yang dimiliki pemain.
skillCards	vector<SkillCard*>	Kartu kemampuan spesial di tangan pemain (maksimal 3).
activeEffects	vector<Effect*>	Daftar efek yang sedang aktif (Shield, Discount, Festival, dll).
controller	PlayerController*	Pointer ke kontroler keputusan
jailTurnsRemaining	int	Sisa giliran percobaan keluar penjara (maksimal 3).
consecutiveDoubles	int	Jumlah dadu double berturut-turut dalam giliran saat ini.
hasUsedSkillCardThisTurn	bool	Penanda agar pemain hanya bisa pakai 1 kartu kemampuan per giliran.
hasRolledDiceThisTurn	bool	Penanda apakah pemain sudah melempar dadu di giliran saat ini.
turnCount	int	Jumlah giliran yang sudah dijalani pemain (untuk MAX_TURN).

2.10.2. Method Kelas

Method	Return	Keterangan
Player(username, startingMoney, controller)	constructor	Membuat pemain baru dengan saldo awal dan kontroler tertentu.
getUsername()	string	Mengembalikan nama pemain.
getMoney()	Money	Mengembalikan saldo uang tunai.
addMoney(amount)	void	Menambah saldo (menggunakan Money::operator+=).
deductMoney(amount)	void	Mengurangi saldo (menggunakan Money::operator-=). Throw InsufficientFundsException jika saldo tidak cukup.
canAfford(amount)	bool	True jika saldo pemain cukup untuk membayar amount.
getPosition()	int	Mengembalikan indeks petak pemain saat ini.
setPosition(pos)	void	Mengubah posisi pemain di papan.
moveBy(steps, boardSize)	void	Memajukan pemain sejauh steps petak (otomatis wrap kalau melewati GO).
getStatus()	PlayerStatus	Mengembalikan status pemain.
setStatus(status)	void	Mengubah status pemain.
isActive()	bool	True jika status pemain ACTIVE.
isBankrupt()	bool	True jika status pemain BANKRUPT.
isJailed()	bool	True jika status pemain JAILED.
getProperties()	vector<PropertyTile*>	Mengembalikan daftar properti yang dimiliki.

addProperty(tile)	void	Menambahkan properti baru ke daftar kepemilikan.
removeProperty(tile)	void	Menghapus properti dari daftar kepemilikan.
getPropertiesByColor(color)	vector<StreetTile*>	Mengembalikan semua street pemain dalam color group tertentu.
hasMonopoly(color)	bool	True jika pemain memonopoli seluruh street dalam color group tertentu.
countRailroads()	int	Menghitung jumlah railroad yang dimiliki.
countUtilities()	int	Menghitung jumlah utility yang dimiliki.
getSkillCards()	vector<SkillCard*>	Mengembalikan kartu kemampuan di tangan.
addSkillCard(card)	void	Menambahkan kartu ke tangan. Throw CardSlotFullExceptionjika sudah 3 kartu.
removeSkillCard(index)	SkillCard*	Menghapus dan mengembalikan kartu di indeks tertentu.
getActiveEffects()	vector<Effect*>	Mengembalikan daftar efek aktif.
addEffect(effect)	void	Menambahkan efek baru ke pemain.
tickEffects()	void	Mengurangi durasi setiap efek; menghapus efek yang sudah expired.
applyOutgoingModifiers(amount)	Money	Menerapkan efek seperti DiscountEffect ke jumlah pembayaran keluar.
applyIncomingModifiers(amount, prop)	Money	Menerapkan efek seperti FestivalEffect ke jumlah penerimaan sewa.
isPaymentBlocked()	bool	True jika pemain sedang dilindungi ShieldEffect.
getController()	PlayerController*	Mengembalikan pointer ke kontroler keputusan.
getTotalWealth()	Money	Menghitung total kekayaan: uang tunai + harga beli semua properti + harga beli semua bangunan.

resetTurnFlags()	void	Reset penanda hasUsedSkillCardThisTurn, hasRolledDiceThisTurn, consecutiveDoubles di awal giliran.
incrementJailTurn()	void	Menambah 1 ke jailTurnsRemaining saat pemain gagal keluar penjara.
operator<(other)	bool	Membandingkan total kekayaan untuk menentukan klasemen akhir (Player A < Player B jika kekayaan A lebih kecil).
operator>(other)	bool	Kebalikan dari <.

2.10.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Money	Player memiliki atribut money: Money dan menggunakan operator Money untuk semua transaksi.
PropertyTile	Player memiliki vector pointer ke PropertyTile* (kepemilikan properti). Sebaliknya, setiap PropertyTile menyimpan pointer balik ke Player sebagai owner.
SkillCard	Player memiliki vector pointer ke SkillCard* di tangan (maksimal 3).
Effect	Player memiliki vector pointer ke Effect* aktif. Effect memodifikasi behavior pembayaran/penerimaan via applyOutgoingModifiers() dan applyIncomingModifiers().
PlayerController	Player menggunakan PlayerController via controller pointer (Strategy Pattern). Player tidak pernah memutuskan sendiri - selalu delegate ke controller.
ColorGroup	Method hasMonopoly() dan getPropertiesByColor() berinteraksi dengan ColorGroup untuk validasi monopoli.
TurnManager	TurnManager menyimpan urutan dan melacak Player mana yang sedang giliran.

BankruptcyManager	Memanggil method Player untuk likuidasi aset (sell/mortgage properti).
MoneyChangedEvent	Setiap perubahan saldo Player memicu publikasi MoneyChangedEvent ke EventBus.
GameStateView	PlayerView (read-only snapshot) dibuat dari Player untuk diberikan ke COM controller dan UI.

2.11. Kelas Board

Merepresentasikan papan permainan Nimonspoli yang terdiri dari kumpulan petak (Tile*). Board berfungsi sebagai container murni untuk semua tile, menyediakan akses cepat berdasarkan indeks maupun kode petak, dan mendukung ukuran dinamis (20-60 petak) untuk memenuhi bonus Papan Dinamis. Board tidak menyimpan logika permainan - semua perilaku petak didelegasikan ke masing-masing Tile melalui polymorphism.

2.11.1. Atribut Kelas

Atribut	Tipe	Keterangan
tiles	vector<Tile*>	Daftar pointer ke seluruh petak di papan, terurut sesuai indeks (0-based, searah jarum jam).
tilesByCode	map<string, Tile*>	Map dari kode petak (misal "JKT", "BDG") ke pointer Tile, untuk akses O(log n).
size	int	Jumlah total petak di papan (40 untuk papan standar, 20-60 untuk papan dinamis).
colorGroups	map<Color, ColorGroup*>	Map dari warna ke ColorGroup yang berisi seluruh street dengan warna tersebut.
goIndex	int	Indeks petak GO (Petak Mulai). Disimpan untuk akses cepat saat menghitung gaji passing GO.
jailIndex	int	Indeks petak Penjara, untuk teleport saat pemain masuk penjara.

2.11.2. Method Kelas

Method	Return	Keterangan
Board(tiles, colorGroups)	constructor	Membuat board dari list tile dan color groups yang sudah dibangun BoardFactory.
~Board()	destructor	Menghapus semua Tile* dan ColorGroup* yang dimiliki.
getTile(index)	Tile*	Mengembalikan pointer ke petak pada indeks tertentu. Throw InvalidPropertyException jika out-of-range.
getTileByCode(code)	Tile*	Mengembalikan pointer ke petak dengan kode tertentu. Throw exception jika tidak ditemukan.
getSize()	int	Mengembalikan jumlah total petak di papan.
getGoIndex()	int	Mengembalikan indeks petak GO.
getJailIndex()	int	Mengembalikan indeks petak Penjara.
getColorGroup(color)	ColorGroup*	Mengembalikan pointer ke ColorGroup berdasarkan warna.
findNearestStation(fromIndex)	int	Mencari indeks stasiun (Railroad) terdekat dari posisi tertentu, searah jarum jam. Digunakan oleh ChanceGoToStation.
getNextTileOfType(fromIndex, type)	int	Mencari indeks petak terdekat dengan tipe tertentu dari posisi tertentu.
getAllStreets()	vector<StreetTile*>	Mengembalikan seluruh StreetTile di papan. Digunakan untuk validasi monopoli dan listing properti.
getAllProperties()	vector<PropertyTile*>	Mengembalikan seluruh properti (Street, Railroad, Utility) di papan.

isValidIndex(index)	bool	True jika indeks berada dalam rentang [0, size).
wrapPosition(position)	int	Membungkus posisi pemain dengan modulo size agar tetap dalam batas papan saat melewati GO.

2.11.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Tile	Board memiliki (komposisi) seluruh Tile* melalui tiles. Bertanggung jawab atas lifetime semua tile.
ColorGroup	Board memiliki (komposisi) seluruh ColorGroup*. Setiap ColorGroup menyimpan referensi balik ke StreetTile yang termasuk warnanya.
BoardFactory	Board dibuat oleh BoardFactory::createBoard() atau createDynamicBoard() berdasarkan konfigurasi. Board sendiri tidak melakukan parsing config.
GameEngine	GameEngine memiliki Board sebagai komponen inti. Engine memanggil getTile(player.getPosition()) untuk menentukan petak tempat pemain mendarat.
Player	Player menyimpan position: int yang merupakan indeks ke salah satu petak di Board.
ChanceCard	ChanceGoToStation memanggil Board::findNearestStation() untuk menentukan tujuan teleport.
BoardFactory	Hanya kelas ini yang dapat membangun Board (Builder Pattern partial).

2.12. Kelas ColorGroup

Merepresentasikan satu set warna yang berisi kumpulan StreetTile dengan warna yang sama (cokelat, biru muda, pink, oranye, merah, kuning, hijau, biru tua). ColorGroup adalah kelas pembantu yang menyentralkan logika monopoli dan validasi pemerataan bangunan, sehingga StreetTile tidak perlu mengetahui detail tile lain dalam color group-nya. Kelas ini memungkinkan validasi kompleks seperti "apakah pemain memonopoli set ini?" atau "apakah selisih bangunan antar petak tidak melebihi 1?" dilakukan dalam satu tempat.

2.12.1. Atribut Kelas

Atribut	Tipe	Keterangan
color	Color	Enum warna color group ini (BROWN, LIGHT_BLUE, PINK, ORANGE, RED, YELLOW, GREEN, DARK_BLUE).
streets	vector<StreetTile*>	Daftar pointer ke seluruh StreetTile yang termasuk dalam warna ini. Berisi 2 atau 3 street sesuai aturan papan.

2.12.2. Method Kelas

Method	Return	Keterangan
ColorGroup(color)	constructor	Membuat color group kosong dengan warna tertentu. Street ditambahkan kemudian via addStreet().
addStreet(street)	void	Menambahkan StreetTile ke color group. Dipanggil oleh BoardFactory saat konstruksi papan.
getColor()	Color	Mengembalikan warna color group ini.
getStreets()	vector<StreetTile*>	Mengembalikan seluruh StreetTile dalam color group.

getStreetCount()	int	Mengembalikan jumlah street dalam color group (2 atau 3).
isMonopolizedBy(player)	bool	True jika seluruh street dalam color group ini dimiliki oleh playeryang sama dan tidak ada yang berstatus MORTGAGED.
getOwner()	Player*	Mengembalikan pemilik tunggal jika dimonopoli, atau nullptr jika belum dimonopoli.
allHaveMinBuildings(level)	bool	True jika setiap street dalam color group memiliki minimal levelbangunan. Digunakan untuk validasi pemerataan saat upgrade ke hotel.
getMaxBuildingDifference()	int	Mengembalikan selisih maksimum jumlah bangunan antar street dalam color group. Harus ≤ 1 untuk validasi pemerataan.
canBuildOn(street)	bool	True jika street boleh dibangun (sudah dimonopoli, tidak digadaikan, dan memenuhi aturan pemerataan).
getBuildableStreets()	vector<StreetTile*>	Mengembalikan daftar street yang memenuhi syarat pemerataan untuk dibangun selanjutnya. Digunakan oleh perintah BANGUN.
hasAnyBuildings()	bool	True jika minimal satu street dalam color group memiliki bangunan. Digunakan untuk validasi gadai.
getTotalBuildingCount()	int	Mengembalikan total jumlah rumah + hotel di seluruh color group.
getTotalBuildingValue()	Money	Mengembalikan total harga beli semua bangunan dalam color group. Digunakan saat menghitung total kekayaan dan likuidasi.
getHalfBuildingValue()	Money	Mengembalikan setengah dari total harga bangunan, untuk perhitungan nilai jual bangunan ke Bank saat gadai/likuidasi.
sellAllBuildings()	Money	Menjual semua bangunan dalam color group ke Bank dan mengembalikan total uang yang diterima. Dipanggil saat pemain ingin menggadaikan properti yang masih ada bangunan dalam color group-nya.

2.12.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
StreetTile	ColorGroup menyimpan vector pointer ke StreetTile*. Setiap StreetTile juga menyimpan referensi balik ke ColorGroup-nya melalui field colorGroup.
Board	Board memiliki (komposisi) semua ColorGroup melalui colorGroups: map<Color, ColorGroup*>. ColorGroup dibuat sekali oleh BoardFactory.
BoardFactory	BoardFactory yang membangun ColorGroup dan menambahkan StreetTile ke dalamnya saat parsing konfigurasi.
Player	Method Player::hasMonopoly(color) mendelegasikan ke ColorGroup::isMonopolizedBy(this).
BuildCommand	Saat eksekusi BANGUN, command memanggil getBuildableStreets() dan canBuildOn() untuk menentukan opsi pembangunan yang valid.
MortgageCommand	Sebelum menggadaikan street, command memanggil hasAnyBuildings(). Jika true, memanggil sellAllBuildings() untuk menjual seluruh bangunan dalam color group terlebih dahulu.
BankruptcyManager	Saat menghitung potensi likuidasi, manager memanggil getHalfBuildingValue() untuk mengetahui nilai jual bangunan.
StreetTile::calculateRent()	Saat menghitung sewa, StreetTile memeriksa colorGroup->isMonopolizedBy(owner) untuk menentukan apakah sewa dasar digandakan dua kali (aturan monopoli tanpa bangunan).

2.13. Kelas TransactionLogger

Mencatat seluruh kejadian signifikan dalam permainan ke dalam log terstruktur yang dapat diakses melalui perintah CETAK_LOG dan disimpan bersama state permainan saat SIMPAN. TransactionLogger bekerja sebagai listener pasif di latar belakang yang berlangganan ke EventBus, setiap kali GameEngine mempublikasikan event (misalnya RentPaidEvent, PropertyAcquiredEvent, BankruptcyEvent), logger otomatis mencatatnya tanpa perlu dipanggil eksplisit. Pendekatan ini memastikan tidak ada satu pun aksi penting yang terlewat dari log dan menjaga GameEngine tetap bersih dari pemanggilan logging manual.

2.13.1. Atribut Kelas

Atribut	Tipe	Keterangan
entries	vector<LogEntry>	Kumpulan seluruh entri log dalam urutan kronologis. Setiap entri menyimpan turn, username, jenis aksi, dan detail.

2.13.2. Method Kelas

Method	Return	Keterangan
TransactionLogger()	constructor	Membuat logger kosong dengan list entries kosong.
log(turn, username, actionType, detail)	void	Menambahkan satu entri baru ke vector entries.
logEntry(entry)	void	Versi overload yang menerima LogEntry jadi langsung dipakai (digunakan saat load dari save file).
onEvent(event)	void	Override dari IEventListener::onEvent(). Menerima GameEvent, mengkonversi ke teks log yang sesuai, lalu memanggil log().
getFullLog()	vector<LogEntry>	Mengembalikan seluruh entri log untuk perintah CETAK_LOG tanpa argumen.
getLastN(n)	vector<LogEntry>	Mengembalikan n entri terakhir untuk perintah CETAK_LOG <n>.
getLogCount()	int	Mengembalikan jumlah total entri log.
clearLog()	void	Mengosongkan seluruh entri log. Digunakan saat New Game dimulai.

serializeForSave()	vector<LogEntry>	Mengembalikan list entries dalam bentuk yang siap di-serialize oleh SaveManager ke file teks.
loadFromSave(entries)	void	Memuat list entries dari file save, menggantikan entries saat ini. Dipanggil oleh LoadCommand.
formatEntry(entry)	string	Helper internal yang memformat satu LogEntry menjadi string yang siap ditampilkan ke UI (misal [Turn 14] Uname3 | SEWA | Bayar M10000 ke Uname2).

2.13.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
LogEntry	TransactionLogger memiliki vector dari struct LogEntry. Setiap LogEntry adalah satuan data terkecil log.
IEventListener	TransactionLogger mengimplementasikan interface IEventListener agar dapat berlangganan ke EventBus. Method onEvent() adalah satu-satunya entry point bagi event eksternal.
EventBus	TransactionLogger berlangganan ke EventBus pada saat inisialisasi GameEngine. EventBus akan memanggil onEvent() setiap kali ada event baru dipublikasikan oleh GameEngine atau manager lain.
GameEvent	TransactionLogger bergantung pada seluruh subclass GameEvent untuk menerjemahkan tipe event menjadi format log yang sesuai. Misalnya RentPaidEvent -> kategori SEWA, PropertyAcquiredEvent -> kategori BELI.
GameEngine	GameEngine memiliki TransactionLogger sebagai komponen inti, dan mendaftarkannya sebagai listener ke EventBus saat startup.
PrintLogCommand	Saat eksekusi CETAK_LOG, command memanggil getFullLog() atau getLastN() lalu mengirim hasilnya ke IGameView::showTransactionLog().

SaveManager	SaveManager memanggil serializeForSave() saat menyimpan game, dan loadFromSave() saat memuat game. Format penyimpanan log mengikuti struktur yang didefinisikan di spesifikasi (<TURN> <USERNAME> <JENIS_AKSI> <DETAIL>).
LoadCommand	Setelah memuat state, command memanggil loadFromSave() untuk merekonstruksi seluruh log dari file.

2.14. Kelas GameState

Snapshot mutable dari seluruh state permainan pada satu titik waktu. GameState menyimpan semua data yang dibutuhkan untuk merekonstruksi permainan secara penuh, termasuk giliran saat ini, daftar pemain, urutan giliran, state seluruh properti, state deck kartu, dan log transaksi. Kelas ini hanya boleh dimodifikasi oleh GameEngine, kelas lain mendapatkan versi read-only melalui GameStateView. GameState menjadi titik integrasi utama untuk fitur Save/Load.

2.14.1. Atribut Kelas

Atribut	Tipe	Keterangan
currentTurn	int	Nomor giliran saat ini (1-based). Bertambah setelah seluruh pemain menyelesaikan satu siklus.
maxTurn	int	Batas maksimum giliran yang dibaca dari misc.txt. Bernilai <1 jika permainan dalam mode bankruptcy-only.
players	vector<Player*>	Daftar pointer ke seluruh pemain (aktif maupun bangkrut).
turnOrder	vector<Player*>	Urutan giliran pemain yang ditentukan saat awal permainan (acak) atau dimuat dari save.
activePlayerIndex	int	Indeks pemain yang sedang giliran aktif di vector turnOrder.
propertyStates	vector<PropertyTile*>	Referensi ke seluruh properti di papan untuk akses cepat saat serialisasi.
skillDeckState	CardDeck<SkillCard>*	Referensi ke deck kartu kemampuan untuk disimpan urutannya saat save.

logEntries	vector<LogEntry>	Salinan seluruh log transaksi yang dimiliki TransactionLogger.
hasRolledDice	bool	Penanda apakah pemain aktif sudah melempar dadu di giliran ini. Mempengaruhi validasi perintah SIMPAN.
hasUsedSkillCard	bool	Penanda apakah pemain aktif sudah menggunakan kartu kemampuan di giliran ini.
consecutiveDoubles	int	Jumlah dadu kembar berturut-turut di giliran aktif saat ini.

2.14.2. Method Kelas

Method	Return	Keterangan
GameState()	constructor	Membuat state kosong dengan nilai default. Diisi oleh GameEngine setelah inisialisasi.
getCurrentTurn()	int	Mengembalikan nomor giliran saat ini.
setCurrentTurn(turn)	void	Mengubah nomor giliran. Dipanggil oleh LoadCommand.
getMaxTurn()	int	Mengembalikan batas maksimum giliran.
getPlayers()	vector<Player*>	Mengembalikan daftar seluruh pemain.
getActivePlayer()	Player*	Mengembalikan pointer ke pemain yang sedang giliran.
getActivePlayerIndex()	int	Mengembalikan indeks pemain aktif di turnOrder.
setActivePlayerIndex(idx)	void	Mengubah indeks pemain aktif. Digunakan saat advance turn dan load.
getTurnOrder()	vector<Player*>	Mengembalikan urutan giliran pemain.

setTurnOrder(order)	void	Mengatur ulang urutan giliran (saat new game atau load).
getLogEntries()	vector<LogEntry>	Mengembalikan seluruh log transaksi.
setLogEntries(entries)	void	Mengatur ulang log saat load game.
getHasRolledDice()	bool	Apakah pemain aktif sudah melempar dadu.
setHasRolledDice(value)	void	Mengubah penanda lempar dadu.
getHasUsedSkillCard()	bool	Apakah pemain aktif sudah pakai kartu kemampuan.
setHasUsedSkillCard(value)	void	Mengubah penanda pakai kartu.
getConsecutiveDoubles()	int	Mengembalikan jumlah dadu kembar berturut-turut.
incrementConsecutiveDoubles()	void	Menambah satu ke jumlah dadu kembar.
resetConsecutiveDoubles()	void	Reset ke 0 saat dadu non-kembar atau giliran berakhir.
resetTurnFlags()	void	Reset hasRolledDice, hasUsedSkillCard, dan consecutiveDoubles di awal giliran baru.
canSave()	bool	True jika pemain aktif belum melakukan aksi apa pun (validasi perintah SIMPAN).
toView()	GameStateView	Membuat snapshot read-only dari state saat ini untuk diberikan ke PlayerController (terutama COM) dan IGameView.

2.14.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi

GameEngine	GameEngine memiliki (komposisi) GameState dan merupakan satu-satunya kelas yang berhak memodifikasinya melalui setter.
Player	GameState menyimpan vector pointer ke Player* melalui players dan turnOrder. Lifetime player dimiliki GameEngine.
LogEntry	GameState menyimpan vector struct LogEntry yang merupakan salinan dari log TransactionLogger pada saat snapshot.
GameStateView	GameState menyediakan method toView() yang menghasilkan GameStateView sebagai snapshot read-only. Hubungan ini adalah conversion, bukan inheritance.
SaveManager	SaveManager membaca seluruh atribut GameState melalui getter untuk diserialisasi ke file teks, dan mengisi GameState melalui setter saat memuat dari file.
TurnManager	TurnManager memodifikasi activePlayerIndex dan currentTurn melalui GameState saat advance turn.
Dice	Setiap kali Dice digunakan, GameEngine memperbarui hasRolledDice dan consecutiveDoubles di GameState.

2.15. Kelas GameStateView

Snapshot immutable read-only dari GameState yang diberikan kepada PlayerController (terutama ComputerController) dan IGameView agar mereka dapat membaca state permainan tanpa risiko memodifikasinya secara tidak sengaja. GameStateView berisi salinan data primitif dan view objects (PlayerView, PropertyView) yang juga read-only. Kelas ini menjadi pemenuhan prinsip encapsulation safety dan Tell-Don't-Ask komponen di luar GameEngine hanya boleh "melihat" state, tidak boleh "mengubah" state.

2.15.1. Atribut Kelas

Atribut	Tipe	Keterangan
currentTurn	const int	Nilai giliran saat ini (read-only).

maxTurn	const int	Batas maksimum giliran (read-only).
activePlayerName	const string	Nama pemain yang sedang giliran.
activePlayerIndex	const int	Indeks pemain aktif di urutan giliran.
playerViews	const vector<PlayerView>	Daftar snapshot read-only seluruh pemain. PlayerView menyimpan username, money, position, status, jumlah properti, dan jumlah kartu, tanpa pointer mutable.
propertyViews	const vector<PropertyView>	Daftar snapshot read-only seluruh properti di papan, lengkap dengan owner name, status, dan level bangunan.
boardSize	const int	Ukuran papan untuk validasi posisi tanpa akses ke Board.
hasRolledDice	const bool	Penanda apakah pemain aktif sudah lempar dadu.
hasUsedSkillCard	const bool	Penanda apakah pemain aktif sudah pakai kartu.

2.15.2. Method Kelas

Method	Return	Keterangan
GameStateView(state)	constructor	Membuat snapshot dari GameState dengan menyalin seluruh data primitif dan membungkus Player / PropertyTile menjadi View objects.
getCurrentTurn()	int	Mengembalikan giliran saat ini.
getMaxTurn()	int	Mengembalikan batas maksimum giliran.
getActivePlayerName()	string	Mengembalikan nama pemain aktif.

getActivePlayerIndex()	int	Mengembalikan indeks pemain aktif.
getPlayers()	vector<PlayerView>	Mengembalikan daftar snapshot pemain.
getMyPlayer(username)	PlayerView	Mengembalikan PlayerView spesifik berdasarkan username. Digunakan COM untuk membaca state dirinya.
getOpponents(username)	vector<PlayerView>	Mengembalikan PlayerView seluruh lawan. Digunakan COM untuk evaluasi strategi.
getProperties()	vector<PropertyView>	Mengembalikan snapshot seluruh properti di papan.
getPropertyByCode(code)	PropertyView	Mengembalikan snapshot properti berdasarkan kode.
getOwnedPropertiesOf(username)	vector<PropertyView>	Mengembalikan daftar properti milik pemain tertentu.
getBoardSize()	int	Mengembalikan ukuran papan.
getHasRolledDice()	bool	Apakah pemain aktif sudah lempar dadu.
getHasUsedSkillCard()	bool	Apakah pemain aktif sudah pakai kartu.

2.15.3. Relasi Kelas

3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameState	GameStateView dibuat oleh GameState::toView(). Hubungan conversion, bukan inheritance. GameStateView tidak menyimpan pointer ke GameState. Semua data disalin agar tetap aman walau GameState berubah.
PlayerView	GameStateView memiliki vector struct PlayerView (read-only snapshot dari Player).

PropertyView	GameStateView memiliki vector struct PropertyView (read-only snapshot dari PropertyTile).
PlayerController	PlayerController menerima GameStateView sebagai parameter pada method-method strategy seperti decideBuyProperty(), decideAuction(), decideBuild().
ComputerController	ComputerController paling banyak menggunakan GameStateView untuk evaluasi strategi (mengintip uang lawan, properti yang sudah dimonopoli, posisi pemain di papan).
HumanController	HumanController menggunakan GameStateView secara minimal, hanya untuk menampilkan informasi ke UI sebelum meminta input manusia.
IGameView	IGameView menerima GameStateView sebagai input untuk method seperti showBoard() dan showTurnInfo() agar tidak perlu pointer mutable.x

2.16. Kelas GameConfig

Menyimpan seluruh nilai konfigurasi permainan yang dibaca dari file-file di folder config/ (property.txt, railroad.txt, utility.txt, tax.txt, special.txt, misc.txt, dan board_layout.txt untuk bonus papan dinamis). GameConfig adalah kelas data murni (data holder) yang menjadi sumber tunggal kebenaran untuk semua nilai yang dapat dikonfigurasi, harga properti, tabel sewa, biaya pajak, gaji GO, denda penjara, saldo awal, dan layout papan. Dengan sentralisasi ini, tidak ada satu pun nilai hardcoded yang tersebar di game logic, sehingga gameplay dapat dimodifikasi sepenuhnya hanya dengan mengubah file teks tanpa perlu rekompilasi.

2.16.1. Atribut Kelas

Atribut	Tipe	Keterangan
properties	map<string, PropertyConfig>	Map dari kode properti (misal "JKT") ke struct PropertyConfig yang berisi nama, jenis, warna, harga lahan, nilai gadai, harga upgrade rumah/hotel, dan tabel sewa per level.
railroadRents	map<int, int>	Map dari jumlah railroad yang dimiliki (1-4) ke biaya sewa yang dikenakan.

utilityMultipliers	map<int, int>	Map dari jumlah utility yang dimiliki (1-2) ke faktor pengali sewa terhadap total dadu.
pphFlat	int	Nilai pembayaran flat untuk Pajak Penghasilan (PPH).
pphPercentage	int	Persentase pembayaran PPH terhadap total kekayaan pemain.
pbmFlat	int	Nilai flat untuk Pajak Barang Mewah (PBM).
goSalary	int	Besaran gaji yang diterima pemain saat berhenti atau melewati petak GO.
jailFine	int	Besaran denda yang harus dibayar untuk keluar dari penjara.
maxTurn	int	Batas maksimum giliran permainan. Bernilai <1 jika permainan dalam mode bankruptcy-only.
startingBalance	int	Saldo awal yang diterima setiap pemain di awal permainan.
boardLayout	vector<TileConfig>	Daftar konfigurasi tile untuk membangun papan. Untuk papan standar berisi 40 entri, untuk papan dinamis bisa 20-60 entri.
configDir	string	Path ke direktori konfigurasi yang digunakan, untuk referensi saat reload.

2.16.2. Method Kelas

Method	Return	Keterangan
GameConfig()	constructor	Membuat objek GameConfig kosong. Diisi oleh ConfigLoader setelah parsing file.
getPropertyConfig(code)	PropertyConfig	Mengembalikan konfigurasi properti berdasarkan kode. Throw exception jika tidak ditemukan.
getAllPropertyConfigs()	map<string, PropertyConfig>	Mengembalikan seluruh konfigurasi properti untuk iterasi.

getRailroadRent(count)	int	Mengembalikan biaya sewa railroad berdasarkan jumlah yang dimiliki pemilik.
getUtilityMultiplier(count)	int	Mengembalikan faktor pengali utility berdasarkan jumlah yang dimiliki pemilik.
getPphFlat()	int	Mengembalikan nilai flat PPH.
getPphPercentage()	int	Mengembalikan persentase PPH.
getPbmFlat()	int	Mengembalikan nilai flat PBM.
getGoSalary()	int	Mengembalikan gaji petak GO.
getJailFine()	int	Mengembalikan denda keluar penjara.
getMaxTurn()	int	Mengembalikan batas maksimum giliran.
getStartingBalance()	int	Mengembalikan saldo awal pemain.
getBoardLayout()	vector<TileConfig>	Mengembalikan layout papan untuk dipakai BoardFactory.
getBoardSize()	int	Mengembalikan jumlah tile di papan (boardLayout.size()).
isBankruptcyMode()	bool	True jika maxTurn < 1, artinya permainan berakhir hanya melalui bankruptcy.
isDynamicBoard()	bool	True jika ukuran papan berbeda dari 40 atau ada file board_layout.txteksplisit.
validate()	bool	Memvalidasi seluruh nilai config: harus ada tepat 1 GO dan 1 Penjara, ukuran papan 20-60, semua properti yang direferensikan layout terdefinisi, dst. Throw InvalidBoardConfigException jika gagal.
setProperties(props)	void	Setter untuk diisi oleh ConfigLoader.
setRailroadRents(rents)	void	Setter untuk diisi oleh ConfigLoader.
setUtilityMultipliers(mults)	void	Setter untuk diisi oleh ConfigLoader.

setTaxConfig(pphFlat, pphPct, pbmFlat)	void	Setter gabungan untuk konfigurasi pajak.
setSpecialConfig(goSalary, jailFine)	void	Setter gabungan untuk konfigurasi petak spesial.
setMiscConfig(maxTurn, startBalance)	void	Setter gabungan untuk konfigurasi lain-lain.
setBoardLayout(layout)	void	Setter untuk layout papan.

2.16.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
ConfigLoader	ConfigLoader membuat dan mengisi GameConfig dengan membaca seluruh file di folder config/. Hubungan satu arah: ConfigLoader -> GameConfig.
PropertyConfig	GameConfig menyimpan map dari struct PropertyConfig yang mendefinisikan setiap properti di papan.
TileConfig	GameConfig menyimpan vector struct TileConfig yang mendefinisikan layout dan jenis setiap petak di papan.
BoardFactory	BoardFactory bergantung pada GameConfig untuk membangun Board, membaca boardLayout untuk menentukan tile apa di indeks berapa, dan properties untuk konfigurasi detail setiap properti.
GameBuilder	GameBuilder menerima GameConfig dari ConfigLoader sebagai input pertama dalam rantai builder, lalu meneruskannya ke BoardFactory dan komponen lain yang memerlukannya.
GameEngine	GameEngine memiliki referensi ke GameConfig untuk mengakses nilai-nilai global seperti getGoSalary(), getJailFine(), getMaxTurn() selama runtime.

TaxTile	PPHTile dan PBMTile menggunakan getPphFlat(), getPphPercentage(), dan getPbmFlat() saat menghitung pajak yang harus dibayar pemain.
GoTile	GoTile menggunakan getGoSalary() untuk menentukan jumlah gaji yang diberikan saat pemain mendarat atau melewati petak.
JailTile	JailTile menggunakan getJailFine() untuk menentukan denda yang harus dibayar pemain saat ingin keluar dari penjara.
RailroadTile	RailroadTile menggunakan getRailroadRent(count) saat menghitung sewa berdasarkan jumlah railroad yang dimiliki pemilik.
UtilityTile	UtilityTile menggunakan getUtilityMultiplier(count) saat menghitung sewa berdasarkan total dadu dan jumlah utility pemilik.
Player	Saat pemain dibuat oleh GameBuilder, saldo awal diambil dari getStartingBalance().
TurnManager	TurnManager menggunakan getMaxTurn() untuk menentukan kapan permainan berakhir karena batas giliran tercapai.

2.17. Kelas Tile

Kelas abstrak (abstract base class) yang menjadi parent untuk seluruh jenis petak di papan permainan Nimonspoli. Tile mendefinisikan kontrak umum yang harus dimiliki setiap petak (indeks, kode, nama, tipe) serta method virtual onLand()yang menjadi entry point polymorphism, saat pemain mendarat di sebuah petak, GameEngine cukup memanggil tile->onLand(player, context) tanpa perlu tahu jenis petaknya. Setiap subclass kemudian mengimplementasikan perilakunya sendiri (membayar sewa, menarik kartu, masuk penjara, dll), sehingga GameEngine tidak perlu berisi switch-case raksasa berdasarkan tipe petak. Tile adalah pemenuhan utama syarat Abstract Class & Virtual Function dalam spesifikasi OOP.

2.17.1. Atribut Kelas

Atribut	Tipe	Keterangan
index	int	Indeks petak di papan (0-based, urut searah jarum jam dari GO).

code	string	Kode unik petak (misal "JKT", "BDG", "GO", "PEN"). Dipakai untuk akses cepat via Board::getTileByCode().
name	string	Nama lengkap petak yang ditampilkan ke UI (misal "Jakarta", "Petak Mulai", "Penjara").
type	TileType	Enum tipe petak (STREET, RAILROAD, UTILITY, CHANCE, COMMUNITY_CHEST, FESTIVAL, TAX_PPH, TAX_PBM, GO, JAIL, FREE_PARKING, GO_TO_JAIL).

2.17.2. Method Kelas

Method	Return	Keterangan
Tile(index, code, name, type)	constructor	Constructor protected, hanya dapat dipanggil oleh subclass.
~Tile()	virtual destructor	Virtual destructor memastikan delete pointer Tile* memanggil destructor subclass yang benar.
getIndex()	int	Mengembalikan indeks petak di papan.
getCode()	string	Mengembalikan kode petak.
getName()	string	Mengembalikan nama lengkap petak.
getType()	TileType	Mengembalikan enum tipe petak.
onLand(player, context)	virtual LandingResult	Pure virtual. Dipanggil saat pemain mendarat di petak ini. Setiap subclass wajib mengimplementasikan perilaku spesifiknya dan mengembalikan LandingResult yang berisi aksi lanjutan untuk GameEngine (BUY_PROMPT, PAY_RENT, DRAW_CARD, dst).
onPass(player, context)	virtual void	Default kosong. Dipanggil saat pemain hanya melewati petak (tidak mendarat). Di-override oleh GoTile untuk memberikan gaji passing GO.

getDescription()	virtual string	Default mengembalikan nama. Dapat di-override subclass untuk deskripsi yang lebih kaya saat ditampilkan ke UI.

2.17.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Board	Board memiliki vector pointer ke Tile*. Bertanggung jawab atas lifetime semua tile.
GameContext	Tile menerima referensi ke GameContext di method onLand(), sehingga dapat mengakses Bank, Dice, FestivalManager, Logger, Config tanpa coupling langsung ke GameEngine.
LandingResult	Tile mengembalikan LandingResult dari onLand() sebagai instruksi untuk GameEngine.
Player	Tile menerima referensi ke Player di onLand() untuk dapat memodifikasi state pemain (posisi, uang, status).
GameEngine	GameEngine memanggil tile->onLand() secara polymorphic tanpa mengetahui tipe konkret petak.

2.18. Kelas PropertyTile

Kelas abstrak yang menjadi parent untuk seluruh petak yang dapat dimiliki oleh pemain (Street, Railroad, Utility). PropertyTile menambahkan atribut kepemilikan, status (BANK/OWNED/MORTGAGED), harga beli, nilai gadai, dan efek festival ke dalam kontrak Tile. Setiap subclass kemudian mengimplementasikan calculateRent() dengan formula sewanya masing-masing.

2.18.1. Atribut Kelas

Atribut	Tipe	Keterangan

owner	Player*	Pointer ke pemain pemilik. Bernilai nullptr jika status BANK.
status	PropertyStatus	Status properti: BANK (belum dimiliki), OWNED (dimiliki normal), atau MORTGAGED (digadaikan).
purchasePrice	Money	Harga beli properti dari Bank, dibaca dari config.
mortgageValue	Money	Nilai uang yang diterima pemain saat menggadaikan properti ke Bank.
festivalMultiplier	int	Pengali sewa akibat efek festival (1, 2, 4, atau 8). 1 berarti tidak aktif.
festivalDuration	int	Sisa giliran efek festival (0-3). 0 berarti tidak aktif.
festivalTimesApplied	int	Jumlah kali festival sudah diaktifkan untuk properti ini (maksimum 3 kali penggandaan).

2.18.2. Method Kelas

Method	Return	Keterangan
PropertyTile(...)	constructor	Constructor protected untuk inisialisasi atribut umum properti.
getOwner()	Player*	Mengembalikan pemilik properti, atau nullptr jika berstatus BANK.
setOwner(player)	void	Mengatur pemilik baru. Dipanggil saat pembelian, lelang, atau transfer akibat bankruptcy.
getStatus()	PropertyStatus	Mengembalikan status properti saat ini.
setStatus(status)	void	Mengubah status properti.
isOwned()	bool	True jika status OWNED.

isMortgaged()	bool	True jika status MORTGAGED.
isAvailable()	bool	True jika status BANK.
getPurchasePrice()	Money	Mengembalikan harga beli.
getMortgageValue()	Money	Mengembalikan nilai gadai.
calculateRent(context)	virtual Money	Pure virtual. Setiap subclass mengimplementasikan formula sewanya sendiri.
getSellValue()	virtual Money	Mengembalikan nilai jual properti ke Bank saat likuidasi. Default mengembalikan purchasePrice. Di-override StreetTile untuk menambah setengah harga bangunan.
applyFestival()	bool	Mengaktifkan atau memperkuat efek festival pada properti ini. Mengembalikan true jika berhasil diperkuat, false jika sudah maksimum.
tickFestival()	void	Mengurangi festivalDuration sebanyak 1. Reset multiplier ke 1 jika durasi mencapai 0.
isFestivalActive()	bool	True jika festivalDuration > 0.
getFestivalMultiplier()	int	Mengembalikan pengali sewa festival saat ini.
onLand(player, context)	override LandingResult	Mengimplementasikan logika umum: jika status BANK → BUY_PROMPT atau AUTO_ACQUIRE; jika OWNED milik orang lain → PAY_RENT; jika MORTGAGED → NOTHING.

2.18.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi

Tile	PropertyTile mewarisi (inheritance) dari Tile.
Player	PropertyTile menyimpan pointer ke owner: Player*. Sebaliknya, Player menyimpan vector PropertyTile* di daftar kepemilikannya.
Money	Atribut purchasePrice dan mortgageValue menggunakan kelas Money.
FestivalManager	FestivalManager berinteraksi dengan PropertyTile melalui applyFestival() dan tickFestival().
StreetTile, RailroadTile, UtilityTile	Tiga subclass konkret yang mewarisi PropertyTile.

2.19. Kelas StreetTile : Kelas PropertyTile

Subclass konkret PropertyTile yang merepresentasikan petak lahan dengan warna (cokelat, biru muda, pink, oranye, merah, kuning, hijau, biru tua). StreetTile adalah satu-satunya jenis properti yang dapat dibangun rumah dan hotel di atasnya, dan memiliki sewa berbeda untuk setiap level bangunan. Sewa juga dipengaruhi oleh status monopoli color group dan efek festival.

2.19.1. Atribut Kelas

Atribut	Tipe	Keterangan
colorGroup	ColorGroup*	Pointer ke color group tempat street ini bernaung.
buildingLevel	int	Level bangunan saat ini: 0 (tanah kosong), 1-4 (jumlah rumah), 5 (hotel).
rentTable	vector<int>	Tabel sewa untuk setiap level bangunan, indeks 0-5.
houseCost	Money	Biaya membangun satu rumah, dibaca dari config.
hotelCost	Money	Biaya upgrade ke hotel, dibaca dari config.

2.19.2. Method Kelas

Method	Return	Keterangan
StreetTile(...)	constructor	Inisialisasi seluruh atribut street dari PropertyConfig.
getColorGroup()	ColorGroup*	Mengembalikan pointer ke color group.
getBuildingLevel()	int	Mengembalikan level bangunan saat ini.
getHouseCost()	Money	Mengembalikan biaya per rumah.
getHotelCost()	Money	Mengembalikan biaya upgrade hotel.
getRentTable()	vector<int>	Mengembalikan seluruh tabel sewa.
isHotel()	bool	True jika buildingLevel == 5.
isMaxedOut()	bool	True jika sudah hotel dan tidak bisa dibangun lagi.
canBuild()	bool	True jika street ini boleh dibangun (delegasi ke colorGroup->canBuildOn(this)).
buildHouse()	void	Menambah satu rumah. Throw exception jika tidak boleh build.
upgradeToHotel()	void	Upgrade ke hotel jika sudah punya 4 rumah.
sellOneBuilding()	Money	Menjual satu tingkat bangunan ke Bank dan mengembalikan setengah harga bangunan.
calculateRent(context)	override Money	Menghitung sewa berdasarkan: level bangunan dari rentTable, status monopoli (×2 jika monopoli tanpa bangunan), dan efek festival (×multiplier).
getSellValue()	override Money	Mengembalikan purchasePrice + setengah dari total harga bangunan.

onLand(player, context)	override LandingResult	Mewarisi logika dasar PropertyTile dengan tambahan rute ke BUY_PROMPT saat status BANK.

2.19.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
PropertyTile	StreetTile mewarisi dari PropertyTile.
ColorGroup	StreetTile menyimpan pointer ke ColorGroup-nya, dan mendelegasikan seluruh validasi pembangunan dan monopoli ke ColorGroup.
BuildCommand	BuildCommand memanggil buildHouse() dan upgradeToHotel() saat eksekusi BANGUN.
FestivalManager	FestivalManager memodifikasi efek festival via method warisan PropertyTile.

2.20. Kelas RailroadTile : Kelas PropertyTile

Subclass konkret PropertyTile untuk petak stasiun. Berbeda dengan Street, Railroad otomatis menjadi milik pemain pertama yang mendarat di petak tersebut tanpa proses pembelian atau lelang. Sewa Railroad dihitung berdasarkan jumlah Railroad yang dimiliki pemilik petak.

2.20.1. Atribut Kelas

Tidak menambah atribut baru selain yang diwarisi dari PropertyTile.

2.20.2. Method Kelas

Method	Return	Keterangan

RailroadTile(...)	constructor	Inisialisasi atribut PropertyTile dengan tipe RAILROAD.
calculateRent(context)	override Money	Menghitung jumlah railroad yang dimiliki owner, lalu lookup ke GameConfig::getRailroadRent(count).
onLand(player, context)	override LandingResult	Jika status BANK, maka AUTO_ACQUIRE (kepemilikan langsung berpindah tanpa prompt). Jika OWNED milik orang lain maka PAY_RENT. Jika MORTGAGED maka NOTHING.

2.20.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
PropertyTile	RailroadTile mewarisi dari PropertyTile.
GameConfig	Bergantung pada GameConfig::getRailroadRent() untuk lookup tarif sewa.
Player	Memanggil Player::countRailroads() untuk menghitung jumlah railroad pemilik.

2.21. Kelas UtilityTile : Kelas PropertyTile

Subclass konkret PropertyTile untuk PLN dan PAM. Sama seperti Railroad, kepemilikan Utility otomatis berpindah ke pemain pertama yang mendarat. Sewa dihitung dari total angka dadu yang dilempar pemain pendarat dikalikan faktor pengali yang bergantung pada jumlah Utility yang dimiliki pemilik.

2.21.1. Atribut Kelas

Tidak menambah atribut baru selain yang diwarisi dari PropertyTile.

2.21.2. Method Kelas

Method	Return	Keterangan
UtilityTile(...)	constructor	Inisialisasi atribut PropertyTile dengan tipe UTILITY.
calculateRent(context)	override Money	Mengambil total dadu dari context.dice.getTotal(), lalu mengalikan dengan GameConfig::getUtilityMultiplier(count) berdasarkan jumlah utility pemilik.
onLand(player, context)	override LandingResult	Jika status BANK maka AUTO_ACQUIRE. Jika OWNED milik orang lain maka PAY_RENT. Jika MORTGAGED maka NOTHING.

2.21.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
PropertyTile	UtilityTile mewarisi dari PropertyTile.
GameConfig	Bergantung pada GameConfig::getUtilityMultiplier() untuk lookup faktor pengali.
Dice	Membaca total dadu terakhir melalui GameContext::dice saat menghitung sewa.
Player	Memanggil Player::countUtilities() untuk menghitung jumlah utility pemilik.

2.22. Kelas ActionTile

Kelas abstrak yang menjadi parent untuk seluruh petak yang memicu aksi spesifik (bukan kepemilikan) saat pemain mendarat di atasnya. ActionTile mencakup petak kartu (Chance, Community Chest), petak Festival, dan petak Pajak (PPH, PBM). Kelas ini tidak menambah atribut signifikan dari Tile, hanya menjadi penanda kategori untuk pengelompokan logis dan memungkinkan polymorphism yang lebih spesifik di masa depan.

2.22.1. Atribut Kelas

Tidak menambah atribut baru.

2.22.2. Method Kelas

Method	Return	Keterangan
ActionTile(...)	constructor	Constructor protected.
onLand(player, context)	virtual LandingResult	Tetap pure virtual. Setiap subclass wajib mengimplementasikan.

2.22.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Tile	Mewarisi dari Tile.
ChanceTile, CommunityChestTile, FestivalTile, TaxTile	Subclass konkret.

2.23. Kelas ChanceTile

Petak kartu Kesempatan. Saat pemain mendarat, sistem otomatis menarik satu kartu dari CardDeck<ChanceCard> melalui CardSystem dan mengeksekusi efeknya.

2.23.1. Atribut Kelas

Tidak menambah atribut baru.

2.23.2. Method Kelas

Method	Return	Keterangan
onLand(player, context)	override LandingResult	Mengembalikan LandingResult{DRAW_CHANCE} agar GameEngine menarik kartu Chance dan mengeksekusinya.

2.23.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
ActionTile	Mewarisi dari ActionTile.
CardSystem	GameEngine mendelegasikan penarikan kartu ke CardSystem setelah menerima LandingResult dari tile ini.

2.24. Kelas CommunityChestTile

Petak kartu Dana Umum. Mekanisme identik dengan ChanceTile, tetapi menarik dari deck Community Chest.

2.24.1. Atribut Kelas

Tidak menambah atribut baru.

2.24.2. Method Kelas

Method	Return	Keterangan
onLand(player, context)	override LandingResult	Mengembalikan LandingResult{DRAW_COMMUNITY}.

2.24.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
ActionTile	Mewarisi dari ActionTile.
CardSystem	Menjadi pemicu penarikan kartu Community Chest.

2.25. Kelas FestivalTile

Petak festival yang memberikan pemain kesempatan memilih satu propertinya untuk dilipatgandakan sewanya selama 3 giliran. Petak ini tidak melakukan modifikasi sendiri, hanya mengembalikan instruksi FESTIVAL_PROMPT agar GameEngine menanyakan ke pemain properti mana yang dipilih.

2.25.1. Atribut Kelas

Tidak menambah atribut baru.

2.25.2. Method Kelas

Method	Return	Keterangan
onLand(player, context)	override LandingResult	Mengembalikan LandingResult{FESTIVAL_PROMPT}.

2.25.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
ActionTile	Mewarisi dari ActionTile.
FestivalManager	GameEngine mendelegasikan ke FestivalManager setelah menerima pilihan properti dari pemain.

2.26. Kelas TaxTile

Kelas abstrak parent untuk petak pajak. Memiliki atribut umum yaitu nilai pajak yang akan ditagih, dan method virtual calculateTax() yang di-override oleh subclass untuk formula pajak masing-masing.

2.26.1. Atribut Kelas

Atribut	Tipe	Keterangan
baseAmount	int	Nilai dasar pajak yang dibaca dari GameConfig (PPH flat atau PBM flat).

2.26.2. Method Kelas

Method	Return	Keterangan
TaxTile(...)	constructor	Constructor protected.
calculateTax(player, choice)	virtual Money	Pure virtual. Subclass mengimplementasikan formula pajaknya.
onLand(player, context)	override LandingResult	Mengembalikan LandingResult{PAY_TAX} dengan referensi ke tile ini.

2.26.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
ActionTile	Mewarisi dari ActionTile.
GameConfig	Bergantung pada untuk membaca nilai pajak.
PPHTile, PBMTile	Subclass konkret.

2.27. Kelas PPHTile : Kelas TaxTile

Subclass TaxTile untuk Pajak Penghasilan. Menawarkan dua opsi pembayaran: flat (nilai tetap dari config) atau persentase dari total kekayaan pemain.

2.27.1. Atribut Kelas

Atribut	Tipe	Keterangan
percentage	int	Persentase pembayaran terhadap total kekayaan, dibaca dari config.

2.27.2. Method Kelas

Method	Return	Keterangan
calculateTax(player, choice)	override Money	Jika choice = FLAT, mka mengembalikan baseAmount. Jika choice = PERCENTAGE maka mengembalikan player.getTotalWealth() * percentage / 100.

getPercentage()	int	Mengembalikan persentase pajak.

2.27.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
TaxTile	Mewarisi dari TaxTile.
Player	Memanggil Player::getTotalWealth() untuk opsi persentase.
ITaxStrategy	PPHTile bergantung pada keputusan pemain via strategy interface ini.

2.28. Kelas PBMTile : Kelas TaxTile

Subclass TaxTile untuk Pajak Barang Mewah. Hanya memiliki satu opsi pembayaran flat tanpa pilihan.

2.28.1. Atribut Kelas

Tidak ada penambahan atribut.

2.28.2. Method Kelas

Method	Return	Keterangan
calculateTax(player, choice)	override Money	Selalu mengembalikan baseAmount tanpa memperhatikan parameter choice.

2.28.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
TaxTile	Mewarisi dari TaxTile.

2.29. Kelas SpecialTile

Kelas abstrak parent untuk petak-petak spesial di papan permainan: GO, Penjara, Bebas Parkir, dan Pergi ke Penjara. Petak-petak ini tidak dimiliki dan tidak mengikuti pola petak aksi standar, masing-masing memiliki perilaku unik.

2.29.1. Atribut Kelas

Tidak ada penambahan atribut.

2.29.2. Method Kelas

Method	Return	Keterangan
SpecialTile(...)	constructor	Constructor protected.
onLand(player, context)	virtual LandingResult	Tetap pure virtual.

2.29.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Tile	Mewarisi dari Tile.

GoTile, JailTile, FreeParkingTile, GoToJailTile	Subclass konkret.

2.30. Kelas GoTile

Petak Mulai (Petak GO). Saat pemain berhenti tepat di petak ini atau melewatinya, pemain menerima gaji sebesar goSalarydari Bank.

2.30.1. Atribut Kelas

Tidak menambah atribut baru.

2.30.2. Method Kelas

Method	Return	Keterangan
onLand(player, context)	override LandingResult	Memberi gaji ke pemain via context.bank.payPlayer() dan mengembalikan LandingResult{NOTHING}.
onPass(player, context)	override void	Memberi gaji passing GO via context.bank.payPlayer().

2.30.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
SpecialTile	Mewarisi dari SpecialTile.
Bank	Memanggil Bank::payPlayer() untuk memberikan gaji.
GameConfig	Membaca nilai gaji dari GameConfig::getGoSalary().

2.31. Kelas JailTile

Petak Penjara. Memiliki dua mode. pemain hanya mampir (tidak ada penalti) atau pemain berstatus tahanan (harus keluar dengan denda, kartu Free Pass, atau dadu kembar).

2.31.1. Atribut Kelas

Tidak ada penambahan.

2.31.2. Method Kelas

Method	Return	Keterangan
onLand(player, context)	override LandingResult	Jika status pemain JAILED, mengembalikan LandingResult{NOTHING} (sudah ditangani GameEngine sebelum memanggil onLand). Jika status ACTIVE, pemain hanya mampir, mengembalikan LandingResult{NOTHING}.

2.31.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
SpecialTile	**Mewarisi** dari SpecialTile.
GameConfig	**Membaca** denda dari GameConfig::getJailFine() saat pemain ingin keluar.

2.32. Kelas FreeParkingTile

Petak Bebas Parkir yang berfungsi sebagai rest area. Tidak ada aksi apa pun saat pemain mendarat.

2.32.1. Atribut Kelas

Tidak menambah atribut baru.

2.32.2. Method Kelas

Method	Return	Keterangan
onLand(player, context)	override LandingResult	Selalu mengembalikan LandingResult{NOTHING}.

2.32.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
SpecialTile	Mewarisi dari SpecialTile.

2.33. Kelas GoToJailTile

Petak jebakan yang langsung memindahkan pemain ke Penjara. Pemain tidak menerima gaji walau melewati GO, dan gilirannya berakhir saat itu juga.

2.33.1. Atribut Kelas

Tidak menambah atribut baru.

2.33.2. Method Kelas

Method	Return	Keterangan

onLand(player, context)	override LandingResult	Mengembalikan LandingResult{GO_TO_JAIL}. GameEngine yang akan memindahkan pemain ke Board::getJailIndex(), mengubah status menjadi JAILED, dan mengakhiri giliran.

2.33.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
SpecialTile	Mewarisi dari SpecialTile.
Board	GameEngine memanggil Board::getJailIndex() untuk teleport pemain.

2.34. Kelas Card

2.34.1. Atribut Kelas

Atribut	Tipe	Keterangan
name	string	Nama yang ditampilkan di antarmuka dan log transaksi.
description	string	Deskripsi efek kartu.

2.34.2. Method Kelas

Method	Return	Keterangan

Card(name, description)	constructor	Inisialisasi kartu.
virtual ~Card()	destructor	Penghancuran objek milik kelas.
getName()	string	Nama kartu.
getDescription()	string	Deskripsi kartu.
virtual getType()	string	Pure virtual tipe subclass Card.

2.34.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
ActionCard	**Inheritance**, menambahkan kontrak execute() untuk Chance dan CommunityChest.
SkillCard	**Inheritance**, menambahkan kontrak activate() untuk kemampuan khusus di tangan player.

2.34.4. Kelas ActionCard : Card

2.34.4.1. Atribut Kelas

Atribut	Tipe	Keterangan
-	-	Hanya atribut turunan.

2.34.4.2. Method Kelas

Method	Return	Keterangan
ActionCard(name, description)	constructor	Inisialisasi objek.
virtual ~ActionCard()	destructor	Penghancuran objek milik kelas.
virtual execute(player, ctx)	CardResult	Pure virtual, dipanggil setelah pemain menarik kartu Chance atau Community.
getType()	string	Return “ActionCard”.

2.34.4.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Card	**Inheritance** dari kelas Card.
ChanceCard	**Subclass**
CommunityChestCard	**Subclass**
GameContext	Parameter untuk execute(player, **ctx**)
CardResult	Hasil dari fungsi execute().
CardSystem	**execute()** dipanggil dalam kelas CardSystem.

2.34.5. Kelas SkillCard

2.34.5.1. Atribut Kelas

Atribut	Tipe	Keterangan
-	-	Hanya atribut turunan.

2.34.5.2. Method Kelas

Method	Return	Keterangan
SkillCard(name, description)	constructor	Inisialisasi objek.
virtual ~ActionCard()	destructor	Penghancuran objek milik kelas.
virtual activate(player, ctx)	CardResult	Pure virtual, dipanggil saat memilih kartu kemampuan pada awal giliran.
virtual getCardType()	SkillCardType	Pure virtual, return jenis kartu.
virtual getSaveValue()	string	Pure virtual, return parameter utama kartu.
virtual getSaveDuration()	string	Pure virtual, return sisa durasi kartu.
getType()	string	Return “SkillCard”.

2.34.5.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Card	**Inheritance** dari kelas Card.
MoveCard	**Subclass**
DiscountCard	**Subclass**
ShieldCard	**Subclass**
TeleportCard	**Subclass**
LassoCard	**Subclass**
DemolitionCard	**Subclass**
Player	Menyimpan dalam vector<SkillCard*>.
CardSystem	Mengelola SkillCard.
SaveManager	Memanggil getSaveValue() dan getSaveDuration()
PlayerController	Memanggil decideSkillCard() dan decideDropCard() yang berkaitan dengan SkillCard.
Effect	Berhubungan dengan SubClass milik SkillCard.

2.35. Kelas Generic CardDeck

2.35.1. Atribut Kelas

Atribut	Tipe	Keterangan

drawCards	vector<T*>	Kartu yang belum ditarik.
usedCards	vector<T*>	Kartu yang telah digunakan.

2.35.2. Method Kelas

Method	Return	Keterangan
CardDeck<T>	constructor	Inisialisasi objek.
~CardDeck<T>	destructor	Penghancuran objek milik kelas.
addCard(card)	void	Menambahkan satu kartu ke drawCards
shuffle()	void	Men-shuffle kartu dalam drawCards.
reshuffleUsed()	void	Memindahkan kartu usedCards ke drawCards dan men-shufflenya.
draw()	T*	Mengambil dan mengembalikan kartu dari drawCards. Jika kosong invokasi reshuffleUsed().
discard(card)	void	Mengembalikan kartu ke usedCards.
isEmpty()	bool	Mengecek isi kedua vector.
getRemainingCount()	int	Mengembalikan jumlah kartu di drawCards.
getTotalCount()	int	Mengembalikan jumlah kartu di kedua vector.
getDrawCards()	const vector<T*>&	Return vector drawCards.
getUsedCards()	const vector<T*>&	Return vector usedCards.

setDrawCards(cards)	void	Mengganti drawCards.
setUsedCards(cards)	void	Mengganti usedCards.

2.35.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
CardSystem	Memanggil addCard() dan reshuffleUsed().
ChanceCard	**Spesialisasi**
CommunityChestCard	**Spesialisasi**
SkillCard	**Spesialisasi**
SaveManager	Memanggil **getDrawCards()** dan **getUsedCards()** saat menyimpan deck.

2.36. Kelas Effect

2.36.1. Atribut Kelas

Atribut	Tipe	Keterangan
remainingTurns	int	Sisa giliran.

2.36.2. Method Kelas

Method	Return	Keterangan
Effect(remainingTurns)	constructor	Inisialisasi objek.
virtual ~Effect()	destructor	Penghancuran objek milik kelas.
virtual onStart	void	Fungsi awal giliran.
virtual onEnd	void	Fungsi akhir giliran.
virtual modifyOutgoingPayment(amount)	Money	Fungsi tiap kali pembayaran.
virtual modifyRent(amount, prop)	Money	Dipanggil applyIncomingModifiers()
virtual blockPayment()	bool	True jika efek memblokir.
isExpired()	bool	Mengecek jika remainingTurns = 0.
getRemainingTurns	int	Mengembalikan remainingTurns.
virtual getDescription()	int	Mengembalikan deskripsi untuk UI.
virtual getEffectType()	string	Mengembalikan tipe efek.

2.36.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi

Player	Memanggil addCard() dan reshuffleUsed().
ShieldEffect	**Subclass**
DiscountEffect	**Subclass**
Money	Digunakan di **modifyOutgoingPayment** dan **modifyRent**.
ShieldCard	Menambahkan ShieldEffect
DiscountCard	Menambahkan DiscountEffect

2.37. Kelas GameEvent

2.37.1. Atribut Kelas

Atribut	Tipe	Keterangan
-	-	-

2.37.2. Method Kelas

Method	Return	Keterangan
virtual ~GameEvent()	destructor	Penghancuran objek milik kelas.
virtual getType()	EventType	Pure virtual, return tipe event.

2.37.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
EventBus	Mengirim GameEvent ke IEventListener.
GameEngine	Mengirim Event melalui EventBus
IEventListener	Menerima GameEvent.
TransactionLogger	Mengubah GameEvent menjadi log.
EventType	Mengembalikan EventType.

2.38. Kelas Command

Command adalah kelas abstrak yang menjadi fondasi pola Command Pattern. Setiap perintah pemain diimplementasikan sebagai subclass tersendiri sehingga penambahan perintah baru cukup membuat satu file baru tanpa mengubah kode yang sudah ada *(Open-Closed Principle)*.

2.38.1. Atribut Kelas

Atribut	Tipe	Keterangan
name	string	Nama perintah (misal "LEMPARDADU", "BANGUN"). Digunakan CommandRegistry untuk memetakan input ke objek Command.

2.38.2. Method Kelas

Method	Return	Keterangan
Command(name)	constructor	Inisialisasi objek milik kelas.
virtual ~Command()	desctructor	Penghancuran objek milik kelas.
execute(engine, player)	void *(pure virtual)*	Menjalankan logika utama perintah. Setiap subclass wajib mengimplementasikan ini.
canExecute(state)	bool *(pure virtual)*	Memvalidasi apakah perintah boleh dijalankan pada kondisi GameStateView saat ini.
getName()	string *(virtual)*	Mengembalikan nama perintah. Digunakan CommandRegistry dan sistem logging.
getHelp()	string *(virtual)*	Mengembalikan teks bantuan singkat cara penggunaan perintah.

2.38.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
CommandRegistry	Menyimpan koleksi Command dan meneruskan eksekusi melalui parseAndExecute.

GameEngine	Diterima sebagai parameter di execute sehingga command dapat memanggil method engine yang diperlukan.
GameStateView	Diterima sebagai parameter di canExecute untuk validasi kondisi permainan secara read-only.
Player	Diterima sebagai parameter di execute sebagai subjek yang menjalankan perintah.

2.39. Kelas PlayerController

Kelas PlayerController mendefinisikan kontrak pengambilan keputusan untuk setiap situasi yang dihadapi pemain.

2.39.1. Atribut Kelas

Atribut	**Tipe**	**Keterangan**
-	-	Kelas abstrak murni; semua state disimpan oleh subkelas konkret.

2.39.2. Method Kelas

Method	Return	Keterangan
chooseCommand(state)	string	Pilih perintah yang akan dijalankan giliran ini.
decideBuyProperty(info, money)	bool	Putuskan apakah membeli properti yang didarati.

decideAuction(currentBid, money)	AuctionAction	Putuskan tawaran atau pass dalam lelang.
decideTax(flatAmount, percentAmount)	TaxChoice	Pilih opsi pajak PPH/PBM yang lebih menguntungkan.
decideSkillCard(cards)	int	Pilih indeks kartu skill yang akan digunakan (-1 = skip).
decideFestivalProperty(props)	string	Pilih kode properti yang mendapat efek festival.
decideBuild(state)	int	Pilih aksi bangun/jual bangunan dari menu.
decideLiquidation(state)	int	Pilih aset yang dijual/digadaikan dalam likuidasi.
decideDropCard(cards)	int	Pilih kartu yang dibuang jika slot penuh.
decideJailPay()	bool	Putuskan apakah membayar denda penjara.
decideTeleportTarget()	string	Pilih kode petak tujuan kartu Teleport.
decideLassoTarget(players)	string	Pilih pemain yang ditarik oleh kartu Lasso.
decideDemolitionTarget(properties)	string	Pilih properti lawan yang dihancurkan kartu Demolition.

2.39.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameEngine	Memanggil chooseCommand() dan decide*() di setiap titik keputusan.
AuctionManager	Memanggil decideAuction() untuk setiap peserta lelang.

BankruptcyManager	Memanggil decideLiquidation() dalam proses likuidasi.
Player	Dimiliki sebagai atribut controller pada setiap objek Player.
HumanController	Subkelas konkret yang mendelegasikan ke IGameInput.
ComputerController	Subkelas konkret yang mengevaluasi GameStateView secara internal.

2.40. HumanController

Kelas HumanController merupakan implementasi PlayerController untuk pemain manusia. Setiap keputusan diteruskan langsung ke IGameInput tanpa logika tambahan, sehingga UI dapat diganti bebas.

2.40.1. Atribut Kelas

Atribut	Tipe	Keterangan
input	IGameInput*	Referensi ke antarmuka input; diinjeksi oleh GameBuilder saat konstruksi.

2.40.2. Method Kelas

Method	Return	Keterangan
chooseCommand(state)	string	Panggil input->getCommand() dan kembalikan hasilnya.
decideBuyProperty(info, money)	bool	Panggil input->getYesNo() untuk konfirmasi pembelian.
decideAuction(currentBid, money)	AuctionAction	Panggil input->getAuctionAction() dari pemain.

decideTax(flat, percent)	TaxChoice	Panggil input->getTaxChoice() untuk pilihan pajak.
decideSkillCard(cards)	int	Panggil input->getSkillCardChoice() dari pemain.
decideLiquidation(state)	int	Panggil input->getLiquidationChoice() dari pemain.
decideDropCard(cards)	int	Panggil input->getDropCardChoice() dari pemain.
decideJailPay()	bool	Panggil input->getJailPayChoice() dari pemain.
decideTeleportTarget()	string	Panggil input->getTeleportTargetChoice() dari pemain.
decideLassoTarget(players)	string	Panggil input->getLassoTargetChoice() dari pemain.
decideDemolitionTarget(properties)	string	Panggil input->getDemolitionTargetChoice() dari pemain.

2.40.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
PlayerController	Mewarisi. Mengimplementasikan semua metode virtual pure.
IGameInput	Dependency, setiap keputusan didelegasikan ke sini.
CLIInput / GUIInput	Implementasi konkret IGameInput yang dapat dipakai.
GameBuilder	Menginstansiasi dan menginjeksi input saat addHumanPlayer().

2.41. ComputerController

Kelas ComputerController merupakan implementasi PlayerController untuk pemain komputer berbasis strategi greedy. Setiap keputusan diambil dengan memilih aksi lokal terbaik berdasarkan kondisi state saat ini, tanpa pencarian mendalam atau simulasi jangka panjang.

2.41.1. Atribut Kelas

Atribut	Tipe	Keterangan
reserveThreshold	int	Batas minimum uang kas yang dipertahankan agar keputusan tetap aman.

2.41.2. Method Kelas

Method	Return	Keterangan
chooseCommand(state)	string	Memilih aksi dengan nilai langsung terbaik, seperti menggunakan skill, membangun, atau melempar dadu.
decideBuyProperty(info, money)	bool	Membeli jika properti menguntungkan dan kas setelah pembelian tetap aman.
decideAuction(currentBid, money)	AuctionAction	Menawar jika properti masih bernilai lebih tinggi daripada bid saat ini menurut heuristik greedy.
decideTax(flatAmount, percentAmount)	TaxChoice	Memilih nilai pajak yang langsung lebih kecil.
decideSkillCard(cards)	int	Menggunakan kartu dengan manfaat langsung terbesar pada situasi sekarang.
decideFestivalProperty(props)	string	Memilih properti milik sendiri dengan potensi kenaikan sewa terbesar.

decideBuild(state)	int	Memilih aksi bangun dengan peningkatan pendapatan lokal terbaik yang masih terjangkau.
decideLiquidation(state)	int	Menjual atau menggadaikan aset dengan kerugian langsung paling kecil terlebih dahulu.
decideDropCard(cards)	int	Membuang kartu dengan utilitas langsung paling rendah.
decideJailPay()	bool	Membayar denda jika keluar penjara memberi keuntungan langsung lebih besar daripada menunggu.
decideTeleportTarget()	string	Memilih petak tujuan dengan manfaat langsung terbesar.
decideLassoTarget(players)	string	Memilih pemain lawan yang paling menguntungkan untuk ditarget saat ini.
decideDemolitionTarget(properties)	string	Memilih properti lawan yang paling merugikan lawan secara langsung jika dihancurkan.

2.41.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
PlayerController	Mewarisi dan mengimplementasikan seluruh metode virtual pure.
GameStateView	Menerima state read-only untuk dievaluasi secara greedy.
GameBuilder	Diinstansiasi saat pemain komputer dibuat.

2.42. Kelas IGameView

IGameView adalah antarmuka abstrak yang memisahkan seluruh output tampilan dari logika permainan. GameEngine dan semua manager tidak pernah memanggil cout langsung, semua output dilakukan melalui method IGameView. Pemisahan ini memungkinkan CLIView dan GUIView ditukar tanpa menyentuh kode logika permainan.

2.42.1.1. Atribut Kelas

Atribut	Tipe	Keterangan
-	-	Seluruh state tampilan dikelola oleh masing-masing implementasi konkret.

2.42.1.2. Method Kelas

Method	Return	Keterangan
showBoard(boardState)	void *(pure virtual)*	Menampilkan papan permainan beserta posisi semua pemain.
showPlayerLanding(playerName, tileName)	void *(pure virtual)*	Mengumumkan pemain mendarat di petak mana.
showPropertyCard(propertyInfo)	void *(pure virtual)*	Menampilkan kartu properti lengkap dengan harga beli, sewa, dan bangunan.
showPlayerProperties(list)	void *(pure virtual)*	Menampilkan daftar properti yang dimiliki seorang pemain.
showBuyPrompt(propertyInfo, playerMoney)	void *(pure virtual)*	Menampilkan prompt pembelian properti beserta saldo pemain.
showRentPayment(rentInfo)	void *(pure virtual)*	Menampilkan informasi pembayaran sewa antara dua pemain.
showTaxPrompt(taxInfo)	void *(pure virtual)*	Menampilkan pilihan pajak flat atau persentase.
showAuctionState(auctionState)	void *(pure virtual)*	Menampilkan status lelang terkini termasuk penawaran tertinggi dan peserta aktif.

showFestivalPrompt(ownedPropertie s)	void *(pure virtual)*	Menampilkan daftar properti pemain untuk dipilih sebagai target festival.
showBankruptcy(bankruptcyInfo)	void *(pure virtual)*	Mengumumkan kebangkrutan seorang pemain.
showLiquidationPanel(liquidationStat e)	void *(pure virtual)*	Menampilkan panel daftar aset yang bisa dijual atau digadaikan.
showCardDrawn(cardInfo)	void *(pure virtual)*	Menampilkan kartu yang baru diambil dari deck.
showSkillCardHand(cards)	void *(pure virtual)*	Menampilkan kartu kemampuan di tangan pemain.
showTransactionLog(entries)	void *(pure virtual)*	Menampilkan isi log transaksi untuk perintah CETAKLOG.
showWinner(winInfo)	void *(pure virtual)*	Menampilkan pemenang beserta klasemen akhir.
showJailStatus(jailInfo)	void *(pure virtual)*	Menampilkan status penjara pemain beserta pilihan aksi keluar.
showMessage(message)	void *(pure virtual)*	Menampilkan pesan generik untuk notifikasi atau konfirmasi sederhana.
showTurnInfo(playerName, turnNum, maxTurn)	void *(pure virtual)*	Menampilkan header giliran berisi nama pemain, nomor giliran, dan batas maksimal.
showBuildMenu(buildMenuState)	void *(pure virtual)*	Menampilkan menu pembangunan rumah/hotel untuk perintah BANGUN.
showMortgageMenu(mortgageMenu State)	void *(pure virtual)*	Menampilkan menu gadai properti.
showRedeemMenu(redeemMenuStat e)	void *(pure virtual)*	Menampilkan menu tebus properti yang sedang digadaikan.

showDropCardPrompt(cards)	void *(pure virtual)*	Menampilkan prompt pemilihan kartu untuk dibuang saat tangan penuh.
showSaveLoadStatus(message)	void *(pure virtual)*	Menampilkan status keberhasilan atau kegagalan operasi simpan/muat.
showDiceResult(d1, d2, playerName)	void *(pure virtual)*	Menampilkan hasil lemparan dadu pemain.

Implementasi konkret:

CLIView: output berbasis terminal dengan kode warna ANSI.

GUIView *(bonus)*: rendering grafis berbasis Raylib/SFML.

2.42.1.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameEngine	Memiliki atribut view bertipe IGameView*. Semua output dalam executeTurn dan handleLanding dipanggil melalui pointer ini.
AuctionManager	Menerima IGameView* sebagai parameter runAuctionLoop untuk menampilkan status lelang.
BankruptcyManager	Memanggil showLiquidationPanel dan showBankruptcy selama proses likuidasi.
CLIView, GUIView	Mengimplementasikan seluruh method pure virtual IGameView.
EventBus	CLIView dan GUIView berlangganan ke EventBus sebagai IEventListener agar update tampilan terjadi secara reaktif.

2.43. Kelas IGameInput

IGameInput adalah antarmuka abstrak untuk semua operasi input pemain. Mekanisme pengambilan input dari terminal maupun GUI disembunyikan dari logika permainan. HumanController menggunakan IGameInput untuk meminta keputusan pemain manusia, sedangkan ComputerController tidak membutuhkannya karena keputusan dihitung secara internal.

2.43.1.1. Atribut Kelas

Atribut	Tipe	Keterangan
(tidak ada)	(tidak ada)	Semua state input dikelola oleh implementasi konkret.

2.43.1.2. Method Kelas

Method	Return	Keterangan
getMenuChoice(options)	int *(pure virtual)*	Menampilkan daftar opsi dan mengembalikan indeks pilihan pemain (0-based).
getCommand()	string *(pure virtual)*	Membaca satu baris perintah dari pemain dan mengembalikannya sebagai string.
getYesNo(prompt)	bool *(pure virtual)*	Menampilkan pertanyaan ya/tidak dan mengembalikan true jika pemain memilih ya.

getNumberInRange(prompt, min, max)	int *(pure virtual)*	Meminta input angka dalam rentang [min, max] dan mengulang sampai input valid.
getString(prompt)	string *(pure virtual)*	Meminta input teks bebas, misalnya nama file saat simpan/muat.
getManualDice()	pair<int,int> *(pure virtual)*	Meminta dua nilai dadu secara manual untuk perintah ATURDADU.
getAuctionAction(playerName, currentBid, playerMoney)	AuctionAction *(pure virtual)*	Menampilkan status lelang dan meminta pemain untuk menawar atau pass.
getTaxChoice()	TaxChoice *(pure virtual)*	Meminta pemain memilih pajak flat atau persentase di petak pajak.
getLiquidationChoice(numOptions)	int *(pure virtual)*	Meminta pemain memilih aset untuk dijual atau digadaikan saat likuidasi.
getSkillCardChoice(numCards)	int *(pure virtual)*	Meminta pemain memilih kartu kemampuan dari tangan.
getPropertyCodeInput(prompt)	string *(pure virtual)*	Meminta input kode properti (misal "JKT") untuk perintah BANGUN, GADAI, atau TEBUS.

2.43.1.3. Relasi Kelas

Kelas Terkait	**Bentuk Relasi**
HumanController	Memiliki atribut bertipe IGameInput*. Setiap method keputusan mendelegasikan ke method IGameInput yang sesuai.
CLIInput, GUIInput	Mengimplementasikan seluruh method pure virtual IGameInput.

AuctionManager	Secara tidak langsung menggunakan IGameInput melalui HumanController::decideAuction yang memanggil getAuctionAction.
BankruptcyManager	Secara tidak langsung menggunakan IGameInput melalui HumanController::decideLiquidation yang memanggil getLiquidationChoice.

2.44. Kelas IEventListener

IEventListener adalah antarmuka abstrak yang menjadi kontrak bagi semua kelas yang ingin menerima notifikasi event dari EventBus. Setiap kali GameEngine mempublikasikan event, EventBus meneruskannya ke seluruh subscriber yang terdaftar. GameEngine cukup mempublikasikan event sekali dan semua subscriber memperbarui diri secara independen.

2.44.1. Atribut Kelas

Atribut	Tipe	Keterangan
(tidak ada)	(tidak ada)	Kelas ini hanya mendefinisikan kontrak method.

2.44.2. Method Kelas

Method	Return	Keterangan
onEvent(event)	void *(pure virtual)*	Dipanggil oleh EventBus setiap ada event baru. Setiap implementasi memproses event sesuai kebutuhannya masing-masing.

2.44.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
EventBus	Menyimpan map dari EventType ke vector IEventListener*. Memanggil onEvent pada semua listener yang berlangganan.
GameEvent	onEvent menerima GameEvent* sebagai parameter. Listener melakukan downcasting ke subclass event yang spesifik.
TransactionLogger	Mengimplementasikan IEventListener. Method onEvent mengkonversi GameEvent menjadi entri log terstruktur.
CLIView / GUIView	Mengimplementasikan IEventListener. Method onEvent menerjemahkan event menjadi update visual.
GameEngine	Mendaftarkan semua subscriber ke EventBus pada saat inisialisasi.

2.45. Kelas IGameRepository

IGameRepository adalah antarmuka abstrak untuk operasi penyimpanan dan pemuatan state permainan. Kelas ini memisahkan apa yang disimpan (GameState) dari bagaimana data itu disimpan. GameEngine hanya mengenal IGameRepository sehingga penggantian format penyimpanan tidak memerlukan perubahan pada logika permainan.

2.45.1. Atribut Kelas

Atribut	Tipe	Keterangan

(tidak ada)	(tidak ada)	Semua mekanisme penyimpanan dikelola oleh implementasi konkret.

2.45.2. Method Kelas

Method	Return	Keterangan
save(state, identifier)	bool *(pure virtual)*	Menyimpan GameState dengan nama identifier tertentu. Mengembalikan true jika berhasil.
load(identifier)	GameState *(pure virtual)*	Memuat dan merekonstruksi GameState dari penyimpanan. Melempar SaveLoadException jika gagal.
exists(identifier)	bool *(pure virtual)*	Mengecek apakah data save dengan identifier tertentu sudah ada.

2.45.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameEngine	Memegang atribut bertipe IGameRepository*. SaveCommand dan LoadCommand mengaksesnya melalui GameEngine.
GameState	Diterima pada operasi save dan dikembalikan pada operasi load sebagai satu-satunya objek yang diserialisasi.

SaveManager	Digunakan oleh TextFileRepository sebagai helper serialisasi dan deserialisasi per-komponen.
SaveCommand	Memanggil save dengan state terkini dari GameEngine saat perintah SIMPAN dieksekusi.
LoadCommand	Memanggil exists lalu load untuk memulihkan state permainan saat perintah MUAT dieksekusi.
SaveLoadException	Dilempar oleh implementasi load jika file tidak ditemukan atau data tidak valid.

Kelas Terkait	**Bentuk Relasi**
GameBuilder	Dipanggil dalam tahap loadConfig() saat membangun permainan.
GameConfig	Menghasilkan objek ini yang kemudian dikonsumsi oleh GameEngine dan BoardFactory.
InvalidBoardConfigException	Dilempar jika format file konfigurasi tidak valid.

2.46. Kelas ConfigLoader

Kelas ConfigLoader bertugas membaca semua file konfigurasi dari direktori config/ dan merakitnya menjadi GameConfig. Semua metode bersifat static, tidak ada state internal.

2.46.1. Atribut Kelas

Atribut	Tipe	Keterangan
-	-	Tidak ada atribut instance. Kelas ini murni kumpulan fungsi transformasi data.

2.46.2. Method Kelas

Method	**Return**	**Keterangan**
loadGameConfig(configDir)	GameConfig	Memanggil semua metode privat dan merakit GameConfig lengkap.
loadProperties(filepath)	map<string, PropertyConfig>	Parsing harga, sewa, dan biaya bangunan per properti.
loadRailroadRents(filepath)	map<int, int>	Parsing tabel sewa kereta berdasarkan jumlah kepemilikan.
loadUtilityMultipliers(filepath)	map<int, int>	Parsing pengganda sewa utilitas berdasarkan jumlah kepemilikan.
loadTaxConfig(filepath)	TaxConfig	Parsing nilai pajak PPH (flat dan persen) dan PBM.
loadSpecialConfig(filepath)	SpecialConfig	Parsing gaji GO, denda penjara, dan konfigurasi khusus lain.
loadMiscConfig(filepath)	MiscConfig	Parsing saldo awal pemain, ronde maksimal, dll.
loadBoardLayout(filepath)	vector<TileConfig>	Parsing urutan dan tipe petak untuk papan dinamis (bonus).

2.46.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameBuilder	Dipanggil dalam tahap loadConfig() saat membangun permainan.
GameConfig	Menghasilkan objek ini yang kemudian dikonsumsi oleh GameEngine dan BoardFactory.
InvalidBoardConfigException	Dilempar jika format file konfigurasi tidak valid.

2.47. Kelas BoardFactory

Kelas BoardFactory bertugas untuk membangun objek Board beserta semua tile dari konfigurasi. Kelas ini mendukung dua mode papan, yaitu papan statis standar dan papan dinamis dari file layout.

2.47.1. Atribut Kelas

Atribut	Tipe	Keterangan
-	-	Tidak ada atribut instance. Pola Abstract Factory diimplementasikan dengan metode static.

2.47.2. Method Kelas

Method	Return	Keterangan

createBoard(config)	Board*	Buat papan standar dari GameConfig.
createDynamicBoard(layout, propertyConfigs)	Board*	Buat papan dari layout kustom (bonus papan dinamis).
createTile(tileConfig, propertyConfigs)	Tile*	Dispatch berdasarkan TileType; instansiasi subkelas Tile yang tepat.
buildColorGroups(board)	map<Color, ColorGroup*>	Kelompokkan StreetTile ke dalam ColorGroup setelah board selesai.
validateBoardLayout(layout)	bool	Pastikan ada tepat 1 GO, 1 JAIL, tanpa kode petak duplikat.

2.47.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameBuilder	Dipanggil dalam tahap buildBoard() saat membangun permainan.
Tile (dan semua subkelas)	Menginstansiasi setiap subkelas tile via createTile().
ColorGroup	Membuat dan menghubungkan ColorGroup ke StreetTile yang sesuai.
GameConfig	Membaca PropertyConfig untuk mengisi data tiap tile properti.
InvalidBoardConfigException	Dilempar jika validasi layout gagal.

2.48. SaveManager

Kelas SaveManager bertugas untuk menangani serialisasi GameState ke file teks dan deserialisasinya kembali. Kelas ini juga memastikan semua kondisi permainan dapat dipulihkan secara akurat.

2.48.1. Atribut Kelas

Atribut	Tipe	Keterangan
-	-	Tidak ada atribut instance. Semua metode bersifat static.

2.48.2. Method Kelas

Method	Return	Keterangan
saveGame(filepath, state)	bool	Konversi GameState ke teks dan tulis ke file.
loadGame(filepath)	GameState	Baca file simpan dan rekonstruksi GameState lengkap.
serializePlayerState(player)	string	Ubah PlayerState menjadi string terstruktur.
deserializePlayerState(data)	PlayerState	Parse string menjadi objek PlayerState.
serializePropertyState(prop)	string	Ubah PropertyState menjadi string terstruktur.
deserializePropertyState(data)	PropertyState	Parse string menjadi objek PropertyState.
serializeDeckState(deck)	string	Simpan urutan draw pile dan discard pile.
deserializeDeckState(data)	DeckState	Pulihkan urutan kartu dari string simpan.

serializeLog(entries)	string	Ubah vector LogEntry menjadi string.
deserializeLog(data)	vector<LogEntry>	Pulihkan daftar log dari string simpan.

2.48.3. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameEngine	Dipanggil saat perintah SIMPAN atau MUAT dieksekusi pemain.
GameState	Sumber data untuk serialize dan target rekonstruksi saat load.
DeckState	Menyimpan dan memulihkan urutan kartu dalam deck.
SaveLoadException	Dilempar jika file tidak ditemukan atau format tidak valid.

2.49. Enums

File Enums bertugas untuk mendefinisikan semua enum class yang digunakan lintas seluruh sistem. Penggunaan enum class mencegah konversi implisit ke integer dan memberikan scoping yang eksplisit.

2.49.1. Atribut Enums

Atribut	Tipe	Keterangan
TileType	enum class	12 jenis petak: STREET, RAILROAD, UTILITY, CHANCE, COMMUNITY_CHEST, FESTIVAL, TAX_PPH, TAX_PBM, GO, JAIL, FREE_PARKING, GO_TO_JAIL.

PropertyStatus	enum class	Status kepemilikan: BANK (belum dimiliki), OWNED, MORTGAGED.
PlayerStatus	enum class	Status pemain: ACTIVE, BANKRUPT, JAILED.
Color	enum class	Kelompok warna jalan: BROWN, LIGHT_BLUE, PINK, ORANGE, RED, YELLOW, GREEN, DARK_BLUE, DEFAULT, GRAY.
SkillCardType	enum class	Jenis kartu skill: MOVE, DISCOUNT, SHIELD, TELEPORT, LASSO, DEMOLITION.
LandingAction	enum class	Hasil onLand(): BUY_PROMPT, PAY_RENT, PAY_TAX, DRAW_CHANCE, DRAW_COMMUNITY, FESTIVAL_PROMPT, GO_TO_JAIL, NOTHING, AUTO_ACQUIRE, BANKRUPT_CHECK.
TaxChoice	enum class	Pilihan pajak: FLAT (nilai tetap) atau PERCENTAGE (persen dari kekayaan).

2.49.2. Relasi Enums

Kelas Terkait	Bentuk Relasi
BoardFactory	Menggunakan TileType untuk dispatch pembuatan tile yang tepat.
PropertyTile	Menyimpan PropertyStatus sebagai atribut status properti.
Player	Menyimpan PlayerStatus; digunakan TurnManager untuk filter pemain aktif.
Tile::onLand()	Mengembalikan LandingAction sebagai bagian dari LandingResult.
CardSystem	Menggunakan SkillCardType untuk identifikasi kartu.

2.50. Kelas GameException

2.50.1. Atribut Kelas

Atribut	Tipe	Keterangan
NimonopoliException	std::exception	Base class semua exception custom; satu catch block dapat menangani semua error permainan.
InsufficientFundsException	NimonopoliException	Dilempar saat saldo tidak cukup. Menyimpan required (Money) dan available (Money).
InvalidCommandException	NimonopoliException	Dilempar saat perintah tidak dikenali atau tidak valid di kondisi saat ini.
InvalidPropertyException	NimonopoliException	Dilempar saat kode properti tidak ditemukan di board.
CardSlotFullException	NimonopoliException	Dilempar saat pemain melebihi batas kartu skill. Menyimpan maxSlots (int).
InvalidBidException	NimonopoliException	Dilempar saat tawaran tidak valid dalam lelang. Menyimpan bidAmount (int) dan reason (string).
SaveLoadException	NimonopoliException	Dilempar saat file tidak ditemukan, format rusak, atau error I/O lain.
InvalidBoardConfigException	NimonopoliException	Dilempar saat konfigurasi papan tidak valid (duplikat tile, GO hilang, dsb.).

2.50.2. Relasi Kelas

Kelas Terkait	Bentuk Relasi
Bank	Melempar InsufficientFundsException saat saldo pemain tidak mencukupi.
AuctionManager	Melempar InvalidBidException untuk penawaran tidak valid.
CardSystem	Melempar CardSlotFullException saat kartu melebihi kapasitas.
SaveManager	Melempar SaveLoadException untuk berbagai kondisi error I/O.
ConfigLoader / BoardFactory	Melempar InvalidBoardConfigException jika konfigurasi tidak valid.
GameEngine	Menangkap semua NimonopoliException di level tertinggi untuk penanganan terpusat.

2.51. Struct

Struct bertugas untuk mendefinisikan struct data sederhana yang dipakai sebagai parameter atau nilai kembalian antar komponen. Struct menegaskan bahwa data ini pasif dan tidak memiliki perilaku sendiri.

2.51.1. Atribut Kelas

Atribut	Tipe	Keterangan
GameContext	struct	Bundel referensi ke Bank, FestivalManager, TransactionLogger, GameConfig, Dice, players, Board, dan currentTurn. Dilewatkan ke Tile::onLand() untuk menghindari coupling langsung dengan GameEngine.
LandingResult	struct	Nilai kembalian dari Tile::onLand(). Berisi LandingAction, amount (int), creditor (Player*), dan property (PropertyTile*).

LogEntry	struct	Satu baris log. Berisi turn (int), username (string), actionType (string), dan detail (string).
AuctionResult	struct	Hasil satu sesi lelang. Berisi winner (Player*), finalPrice (int), dan property (PropertyTile*).
FestivalEffect	struct	Efek festival aktif pada properti. Berisi owner (Player*), multiplier (int: 2/4/8), turnsRemaining (int: 1-3), dan timesApplied (int: maks 3).

2.51.2. Relasi Kelas

Kelas Terkait	Bentuk Relasi
GameContext	Dipakai oleh semua subkelas Tile dalam onLand() untuk mengakses komponen inti.
LandingResult	Dikembalikan oleh Tile::onLand() dan diproses oleh GameEngine::handleLanding().
LogEntry	Disimpan dalam vector<LogEntry> milik TransactionLogger.
AuctionResult	Dikembalikan oleh AuctionManager::startAuction() ke GameEngine.
FestivalEffect	Disimpan dalam activeEffects map milik FestivalManager.

3. Justifikasi

3.1. Alasan Pemilihan Desain

3.1.1. Di sini dipakai untuk Card, Command, Effect, dan Tile karena setiap base class mewakili behavior umum yang di-*share* oleh banyak subclass. Dengan polymorphism, GameEngine bisa process semua tile types dengan cara yang sama (onLand method), CommandRegistry bisa execute semua commands dengan execute method, dan Effect system bisa apply berbagai efek secara uniform. Ini menghilangkan long if-else chains dan membuat penambahan type baru sangat mudah, jadi cukup create subclass baru tanpa modifikasi existing code.

3.1.2. Smart Pointers & Rule of Five. Digunakan untuk memory safety dan clear ownership semantics. Setiap class dengan unique_ptr members (Card, Player, GameEngine, Board) menerapkan Rule of Five dengan deleted copy operations dan default move operations. Ini mencegah shallow copying bugs dan memungkinkan efficient resource transfer via move semantics. unique_ptr automatically cleans up memory ketika object destroyed, eliminating memory leaks.

3.1.3. Player memiliki vector<Effect> bukan inherit dari Effect class, karena composition memberikan lebih banyak flexibility, player bisa punya multiple effects active bersamaan. Ini juga maintains loose coupling. Player tidak perlu tahu tentang specific effect implementations, hanya bahwa effect punya methods seperti modifyRent() dan tick(). Arsitektur ini mudah di-extend dengan effect types baru tanpa touching Player class.

3.2. Kelebihan dan Kekurangan

3.2.1. Kelebihannya yaitu inheritance hierarchy memungkinkan reusability code dan extensibility tanpa perlu memodifikasi logis yang sudah ada sebelumnya. Polymorphism mengeliminasi massive if-else chains, sehingga code jadi lebih clean dan lebih mudah untuk di-maintain. Smart pointers provide memory safety dan explicit ownership semantics. Exception handling (custom NimonopoliException hierarchy) enables centralized error handling. Operator overloading (Money +, -, <, >, Player comparisons) membuat syntax natural dan intuitive.

3.2.2. Kekurangan virtual functions di sini digunakan sebagai slight performance overhead (vtable lookups), negligible untuk pengembangan di sini. Object graph yang bisa dibilang lumayan kompleks (GameEngine -> Board -> Tiles, Players -> Properties) membutuhkan careful lifetime management untuk menghindari dangling pointers. Templates (CardDeck<T>) bisa memnambahkan compilation time overhead. Exception throwing punya biaya performa yang non-trivial, jadi tidak boleh digunakan untuk normal control flow.

3.2.3. Terkait trade-offsnya, vvirtual function overhead di-balance oleh code clarity dan maintainability. Smart pointer complexity kembali di-balance oleh memory safety dan elimination of memory leaks. Object lifetime complexity di-manage dengan clear ownership hierarchy (GameEngine punya Board, Board punya Tiles, Player juga puna references).

3.3. Tantangan Terbesar dalam Pemilihan Desain OOP

Tantangan terbesar adalah menentukan batas tanggung jawab antar kelas secara tepat. Keputusan seperti apakah Tile yang mengeksekusi efek pendaratan atau GameEngine, serta apakah PlayerController menggunakan multiple inheritance dari banyak interface strategi, tidak punya jawaban yang hitam-putih. Akhirnya dipilih pendekatan di mana Tile hanya menghitung dan mengembalikan hasil tanpa mengeksekusi efek apapun, dan PlayerController mengintegrasikan semua interface strategi. Kompromi ini menjaga setiap komponen tetap bisa diuji secara terisolasi, meski dengan konsekuensi hierarki kelas yang sedikit lebih dalam dari yang sebenarnya dibutuhkan.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

4.1.1. Kelas Card

1. Seluruh Subclass Card : Card

Inheritance	Polymorphism	Snippet

Digunakan agar tiap subclass memiliki kontrak yang sama sehingga dapat diproses dengan cara yang sama. Pada kelas Card setiap subclassnya akan memiliki atribut *name* dan *description*.	Digunakan agar fungsi dari tiap subclass dapat berbeda sehingga pada beberapa subclass dapat memiliki fungsi yang sama namun dengan return yang berbeda. Contohnya *getType()*.	`class ActionCard : public Card {`
		`  public:`
		`  ActionCard(std::string name, std::string description);`
		`  ~ActionCard() override;`
		
		`  virtual CardResult execute(Player& player, GameContext& context) = 0;`
		`  std::string getType() const override;`
		`};`

4.1.2. Kelas Command

1. Seluruh Subclass Command : Command

Inheritance	Polymorphism	Snippet

Digunakan agar tiap subclass command memiliki atribut yang sama sehingga dapat diproses dengan cara yang sama. Pada hal ini subclass seperti RollDiceCommand dari Command akan memiliki atribut *name* dan *help* seperti subclass lainnya.	Digunakan agar tiap subclass dapat melaksanakan tanggung jawabnya melalui suatu fungsi yang sama dengan isi yang berbeda. Contohnya tiap subclass dapat memiliki *execute()* dengan hasil yang berbeda.	`class RollDiceCommand : public Command {`
		`public:`
		`  RollDiceCommand();`
		
		`  void execute(GameEngine& engine, Player& player) override;`
		`  bool canExecute(const GameStateView& state) const override;`
		`};`

4.1.3. Kelas Effect

1. Seluruh Subclass Effect : Effect

Inheritance	Polymorphism	Snippet

Memastikan agar tiap subclass memiliki kontrak yang sama. Dalam kasus ini Effect harus memiliki *remainingTurns*.	Banyak efek harus dijalankan dengan cara yang sama dan dengan efek yang berbeda-beda sehingga dibutuhkan suatu fungsi yang sama dengan hasil yang berbeda untuk tiap subclass. Misalnya: *modifyRent()*.	`class DiscountEffect : public Effect {`
		`public:`
		`  DiscountEffect(int percent, int turns);`
		`  Money modifyOutgoingPayment(const Money& amount) const override;`
		`  std::string getDescription() const override;`
		`  std::string getEffectType() const override;`
		
		`private:`
		`  int percent;`
		`};`

4.1.4. Kelas Tile

1. Seluruh Subclass Tile : Tile

Inheritance	Polymorphism	Snippet
Memastikan agar tiap subclass memiliki *code*, *name*, *type* sehingga dapat diproses secara seragam.	Tiap subclass tile membutuhkan proses ketika player menyentuh tile tersebut dan tiap tile memberikan proses yang berbeda-beda dengan satu fungsi yang sama.	

```
class Tile {
public:
    Tile(int id, const std::string& code, const std::string& name, TileType type);
    virtual ~Tile() = default;

    int getId() const;
    const std::string& getCode() const;
    const std::string& getName() const;
    TileType getType() const;

    virtual void onLand(class Player& player, class GameContext& ctx);

private:
    int id;
    std::string code;
    std::string name;
    TileType type;
};
```

4.2. Method/Operator Overloading

4.2.1. Money

Operator Overloading	Snippet
Operator overloading ini digunakan untuk membantu operasi kalkulasi dan perbandingan Money dikarenakan natural dan konsisten sehingga mengurangi redundansi.	

```
class Money {
    public:
        explicit Money(int amount = 0);

        int getAmount() const;
        void setAmount(int value);

        bool isZero() const;
        bool isPositive() const;

        Money operator+(const Money& other) const;
        Money operator-(const Money& other) const;

        Money& operator+=(const Money& other);
        Money& operator-=(const Money& other);

        bool operator<(const Money& other) const;
        bool operator>(const Money& other) const;
        bool operator<=(const Money& other) const;
        bool operator>=(const Money& other) const;
        bool operator==(const Money& other) const;
        bool operator!=(const Money& other) const;
```

4.2.2. Player

Operator Overloading	Snippet

Operator overloading digunakan untuk membandingkan Player berdasarkan *money* yang dimiliki oleh mereka. Hal ini dilakukan agar mengurangi redundansi dan mempermudah operasi natural.	`class Player {`
	`  bool operator<(const Player& other) const;`
	`  bool operator>(const Player& other) const;`
	
	`private:`
	`  std::string username;`
	`  Money money;`

4.2.3. TransactionLogger

Method Overloading	Snippet

Method overloading digunakan untuk mempermudah penggunaan fungsi tanpa membuat fungsi baru untuk mencatat dengan parameter berbeda.	

```
class TransactionLogger : public IEventListener {
public:
    void onEvent(const GameEvent& event) override;

    void log(int turn, const std::string& username, const std::string&
    void logEntry(const LogEntry& entry);
    const std::vector<LogEntry>& getFullLog() const;
    std::vector<LogEntry> getLastN(int n) const;
    int getLogCount() const;
    std::vector<LogEntry> serializeForSave() const;
    void loadFromSave(const std::vector<LogEntry>& loadedEntries);

    void log(const std::string& entry);
```

*Terdapat juga method overloading lainnya seperti operator= pada beberapa class lainnya.

4.3. Template & Generic Classes

4.3.1. CardDeck<T>

Template & Generic	Snippet

CardDeck<T> dijadikan sebagai template agar mempermudah pemakaian ulang berbagai jenis deck sehingga bersifat *type-safe* dan ownership kartu aman.	

```
template <typename T>
class CardDeck {
    public:
        using CardPtr = std::unique_ptr<T>;

        CardDeck() = default;
        ~CardDeck() = default;

        CardDeck(const CardDeck& other) = delete;
        CardDeck& operator=(const CardDeck& other) = delete;

        CardDeck(CardDeck&& other) noexcept = default;
        CardDeck& operator=(CardDeck&& other) noexcept = default;

        void addCard(CardPtr card) {
            if (!card) {
                throw std::invalid_argument("Cannot add null card to c
            }
            drawCards.push_back(std::move(card));
        }

        template <typename U, typename... Args>
        U& emplaceCard(Args&&... args) {
            static_assert(std::is_base_of<T, U>::value, "U must inheri
            auto card = std::make_unique<U>(std::forward<Args>(args)..
            U& reference = *card;
            addCard(std::move(card));
            return reference;
        }

        void shuffle() {
            static std::random_device randomDevice;
            static std::mt19937 generator(randomDevice());
            std::shuffle(drawCards.begin(), drawCards.end(), generator
```

4.4. Exception

4.4.1. NimonopoliException & Subclass

Exception	Snippet

Exception seperti NimonopoliException dan subclassnya InvalidCommandException, CardSlotFullException, dan lainnya direalisasikan agar mempermudah error handling dari program sehingga tidak akan crash ketika sedang berjalan.	

```
class NimonopoliException : public exception {
public:
    explicit NimonopoliException(string message);

    const char* what() const noexcept override;

private:
    string message;
};

Jordhy Branenda, last week | 1 author (Jordhy Branenda)
class InsufficientFundsException final : public NimonopoliException
public:
    InsufficientFundsException(Money required, Money available);

    Money getRequired() const;
    Money getAvailable() const;

private:
    Money required;
    Money available;
};

Jordhy Branenda, 2 days ago | 1 author (Jordhy Branenda)
class InvalidCommandException final : public NimonopoliException {
public:
    explicit InvalidCommandException(string message);
};

Jordhy Branenda, 2 days ago | 1 author (Jordhy Branenda)
class InvalidPropertyException final : public NimonopoliException {
public:
    explicit InvalidPropertyException(string message);
};
```

4.5. C++ Standard Template Library

4.5.1. std::unordered_map

STL	Snippet

std::unordered_map digunakan sebagai storage untuk mempermudah akses command dengan lookup hash map yang cepat.	`class CommandRegistry {`
	`private:`
	`void registerDefaultCommands();`
	`static std::string normalizeName(const std::string& value);`
	
	`std::unordered_map<std::string, std::unique_ptr<Command>> commands;`
	
	`protected:`
	`};`

4.5.2. std::unique_ptr

STL	Snippet

std::unique_ptr digunakan sebagai identifier ownership command sekaligus menghindari memory leak. STL ini juga digunakan pada Card.	

```
class CommandRegistry {
private:
    void registerDefaultCommands();
    static std::string normalizeName(const std::string& value);

    std::unordered_map<std::string, std::unique_ptr<Command>> commands;

protected:
};
```

4.5.3. std::vector

STL	Snippet
std::vector digunakan untuk menyimpan level sewa / *rentLevels* dari tile property karena dapat dimanipulasi dan dikelola secara dinamis dan cepat. STL ini juga digunakan pada TransactionLogger.	

```
class StreetTile : public PropertyTile {
    Money hotelCost;
    std::vector<int> rentLevels;
    int buildingLevel;
    bool monopolyComplete;
};
```

4.5.4. Algoritma STL

STL	Snippet
Algoritma STL seperti std::move dan std::shuffle digunakan untuk mempermudah implementasi dibandingkan manual.	

```
template <typename U, typename... Args>
U& emplaceCard(Args&&... args) {
    static_assert(std::is_base_of<T, U>::value, "U must inherit f
    auto card = std::make_unique<U>(std::forward<Args>(args)...);
    U& reference = *card;
    addCard(std::move(card));
    return reference;
}

void shuffle() {
    static std::random_device randomDevice;
    static std::mt19937 generator(randomDevice());
    std::shuffle(drawCards.begin(), drawCards.end(), generator);
```

4.6. Konsep OOP lain

4.6.1. Abstract Base Class

Abstract Base Class	Snippet

Kelas seperti IEventListener memastikan bahwa ia tidak dapat direalisasikan secara langsung tetapi wajib melalui subclassnya.	`class IEventListener {`
	`public:`
	`  virtual ~IEventListener();`
	`  virtual void onEvent(const GameEvent& event) = 0;`

4.6.2. Composition (Has-A)

Composition	Snippet

Player mengandung Effect (bukan *inheritance*), hal ini membuatnya memiliki coupling yang rendah dan bersifat modular.	

```
class Player {
    void setConsecutiveDoubles(int n);

    int getTurnCount() const;
    void incrementTurnCount();

    bool operator<(const Player& other) const;
    bool operator>(const Player& other) const;

private:
    std::string username;
    Money money;
    int position;
    PlayerStatus status;
    std::vector<PropertyTile*> properties;
    std::vector<std::unique_ptr<SkillCard>> skillCards;
    mutable std::vector<SkillCard*> skillCardView;
    std::vector<std::unique_ptr<Effect>> activeEffects;
    mutable std::vector<Effect*> activeEffectView;
    PlayerController* controller;
    int jailTurnsRemaining;
    int consecutiveDoubles;
    bool hasUsedSkillCardThisTurn;        You, 2 days ago
    bool hasRolledDiceThisTurn;
    int turnCount;
};
```

4.6.3. Rule of Five

Rule of Five	Snippet

GameEngine, Board, Card, dan kelas lainnya menerapkan rule ini agar mencegah shallow-copy state besar dan juga membuat pemindahan objek lebih efisien.	`class GameEngine {`
	`public:`
	`  explicit GameEngine(GameState initialState = GameState());`
	`  GameEngine(const GameEngine& other) = delete;`
	`  GameEngine& operator=(const GameEngine& other) = delete;`
	`  GameEngine(GameEngine&& other) noexcept = default;`
	`  GameEngine& operator=(GameEngine&& other) noexcept = default;`

5. Bonus Yang dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. GUI

Antarmuka grafis NIMONSPOLY dibangun di atas library Raylib dengan arsitektur berbasis interface yang memisahkan logika rendering dari logika permainan. Dua interface utama, IGameView dan IGameInput, bertindak sebagai kontrak abstrak antara GameEngine dengan lapisan presentasi. Implementasi konkret GUIView dan GUIInput menangani seluruh operasi Raylib untuk render papan, animasi dadu, tampilan kartu skill, prompt beli/lelang, hingga input pemain tanpa satu baris pun perubahan yang perlu dilakukan di engine Pendekatan ini mengikuti Strategy Pattern sehingga library grafis dapat diganti tanpa menyentuh logika permainan. Manajemen sumber daya grafis dipusatkan melalui kelas AssetManager yang menerapkan Singleton Pattern. AssetManager memuat seluruh font dan tekstur satu kali saat startup, lalu menyediakannya ke seluruh komponen GUI melalui satu titik akses. Ini mencegah duplikasi resource di memori dan memastikankonsistensi aset di seluruh layar. Alur antar layar diatur oleh enum AppScreen yang mencakup tujuh state: Landing, Setup Jumlah Pemain,Kustomisasi Pemain, Kustomisasi Peta, Load Game, In-Game, dan Game Over. Pergantian layar ditangani oleh GameApp sebagai controller utama loop aplikasi.

Setiap layar GUI diimplementasikan sebagai komponen terpisah dalam subfolder src/views/gui/pages/, masing-masing menangani rendering dan input untuk satu state AppScreen. Layar in-game terdiri dari beberapa subkomponen, GUIViewBoard untuk render papan dan bidak, GUIViewDraw untuk overlay informasi pemain dan properti, serta TileRenderer untuk render tiap tile secara individual. Desain modular ini memungkinkan setiap bagian UI dikerjakan dan diuji secara mandiri, serta memudahkan penambahan layar baru di masa mendatang tanpa mengubah struktur yang sudah ada.

5.1.2. COM

Pendekatan:

Pengendali komputer diimplementasikan melalui ComputerController yang meng-extend PlayerController (Strategy Pattern). AI tidak bertindak random; ia memiliki sistem penilaian (scoring) untuk setiap keputusan:

- Property Scoring: Setiap tile diberi skor berdasarkan kode lokasi, tipe (Street/Railroad/Utility), color group, rasio sewa/harga, dan level bangunan. Orange group (MAL, SMG, SBY) mendapat skor tertinggi karena frekuensi kunjungan tinggi.
- Cash Reserve Management: AI menjaga cadangan uang proporsional terhadap total kekayaan. Jika uang <= 300, reserve = 25-75; jika > 1200, reserve = max(200, cash/5). Ini mencegah AI bangkrut karena pembelian impulsif.
- Auction Budgeting: AI menghitung anggaran maksimal untuk lelang = min(cash - reserve, cash x 45%). Jika cash <= 2000, budget dibatasi 420.
- Liquidation Optimization: Saat likuidasi, AI memilih opsi dengan waste paling kecil. Jika tidak ada yang pas, AI memilih opsi dengan nilai tertinggi sebagai fallback.
- Turn Memory: ComputerController menyimpan snapshot GameStateView dari turn sebelumnya untuk mencegah loop.

Cuplikan Kode:

```
// include/controllers/ComputerController.hpp
class ComputerController final : public PlayerController {
public:
    string chooseCommand(const GameStateView& state) override;
    bool decideBuyProperty(const PropertyInfo& info, Money money) override;
    AuctionDecision decideAuction(int currentBid, Money money) override;
    int decideBuild(const BuildMenuState& state) override;
    int decideLiquidation(const LiquidationState& state) override;
```

```
private:
    int cashReserve(int cash) const;
    int propertyScore(const PropertyInfo& info) const;
    int propertyCodeScore(const string& code) const;
    int cardScore(const CardInfo& card) const;
    void refreshTurnMemory(const GameStateView& state);
};
```

```
// src/controllers/ComputerController.cpp -- property scoring
int ComputerController::propertyScore(const PropertyInfo& info) const {
    int score = propertyCodeScore(info.code);
    switch (info.type) {
        case TileType::STREET:   score += 15; break;
        case TileType::RAILROAD: score += 20; break;
        case TileType::UTILITY:  score += 5;  break;
        default: break;
    }
    switch (info.colorGroup) {
        case Color::ORANGE:     score += 30; break;
        case Color::RED:        score += 28; break;
        case Color::LIGHT_BLUE: score += 24; break;
        case Color::YELLOW:     score += 22; break;
        default: break;
    }
    const int price = info.purchasePrice.getAmount();
    const int bestRent = maxRent(info);
    if (bestRent > 0) {
        score += min(35, bestRent / 40);
        if (price > 0) score += min(25, bestRent * 10 / price);
    }
    score += info.buildingLevel * 20;
    if (info.status == PropertyStatus::MORTGAGED) score -= 35;
    return score;
}
```

```
// Auction AI dengan budgeting cerdas
AuctionDecision ComputerController::decideAuction(int currentBid, Money money) {
    const int cash = money.getAmount();
```

```
    const int minimumBid = max(0, currentBid + 1);
    if (cash <= minimumBid) return AuctionDecision{AuctionAction::PASS, 0};

    int budget = cash - cashReserve(cash);
    budget = min(budget, cash * 45 / 100);
    if (cash <= 2000) budget = min(budget, 420);
    if (budget < minimumBid) return AuctionDecision{AuctionAction::PASS, 0};

    const int step = max(10, cash / 50);
    int bid = currentBid < 0 ? min(budget, max(50, step * 3))
                             : min(budget, currentBid + step);
    bid = max(bid, minimumBid);
    return AuctionDecision{AuctionAction::BID, bid};
}
```

5.1.3. Papan Dinamis

Fitur papan dinamis memungkinkan pemain memilih ukuran papan permainan sebelum permainan dimulai. Ukuran papan yang tersedia berada pada rentang 20 hingga 60 tile dengan kelipatan 4. Dengan adanya fitur ini, permainan tidak hanya bergantung pada satu ukuran papan tetap, tetapi dapat menyesuaikan jumlah tile sesuai pilihan pemain. Konfigurasi untuk setiap ukuran papan disimpan secara terpisah di dalam subfolder config/{ukuran}/. Sebagai contoh, konfigurasi papan berukuran 20 tile disimpan pada config/20/, papan 36 tile pada config/36/, dan seterusnya hingga config/60/. Setiap subfolder berisi tiga file utama, yaitu property.txt, aksi.txt, dan board.txt.

File property.txt berisi data properti yang digunakan pada ukuran papan tersebut, seperti harga lahan, nilai sewa, dan warna grup. File aksi.txt berisi data tile non-properti, seperti Go, Penjara, Kesempatan, Dana Umum, dan Festival. Sementara itu, file board.txt menentukan urutan susunan tile pada papan, dengan satu kode tile dituliskan pada setiap baris. Beberapa file konfigurasi yang bersifat umum, seperti railroad.txt, utility.txt, tax.txt, dan misc.txt, tetap disimpan di root folder config/ karena berlaku untuk seluruh ukuran papan.

Saat permainan dimulai, ukuran papan yang dipilih pemain akan diteruskan ke fungsi GameEngine::loadConfiguration(directory, numTiles). Fungsi ini membentuk path menuju folder konfigurasi sesuai ukuran papan, yaitu config/{numTiles}/, kemudian meneruskannya ke GameBuilder::buildDefaultBoard. Selanjutnya, builder membaca file board.txt untuk mendapatkan urutan tile yang harus dibentuk. Berdasarkan kode tile tersebut, builder akan membuat objek Tile yang sesuai menggunakan data dari property.txt dan aksi.txt.

Board yang terbentuk kemudian diisi secara dinamis melalui Board::addTile(unique_ptr<Tile>). Dengan pendekatan ini, jumlah dan susunan tile tidak lagi ditentukan secara hardcoded di dalam program. Ukuran papan juga diperoleh langsung dari jumlah tile yang tersimpan, yaitu melalui Board::getSize() yang mengembalikan tiles.size(). Hal ini membuat logika permainan, seperti perhitungan posisi pemain, iterasi tile, serta proses save dan load, dapat menyesuaikan diri secara otomatis terhadap ukuran papan yang sedang digunakan. Rendering papan juga dibuat adaptif. Komponen GUIViewBoard menggambar papan berdasarkan jumlah tile aktual dari board.getSize(). Dengan demikian, tampilan papan dapat menyesuaikan ukuran yang dipilih pemain, baik untuk papan kecil maupun besar.

Untuk menjaga validitas konfigurasi, sistem juga melakukan pengecekan terhadap keberadaan folder dan kelengkapan file konfigurasi. Jika subfolder untuk ukuran tertentu tidak ditemukan atau file konfigurasi tidak lengkap, engine akan melempar InvalidBoardConfigException. Mekanisme ini mencegah papan permainan terbentuk dalam kondisi tidak valid dan membantu memastikan konfigurasi yang digunakan sudah sesuai sebelum permainan dijalankan.

5.2. Bonus Kreasi Mandiri

5.2.1. Tile dapat diklik secara langsung

Setiap tile pada papan dapat diklik oleh pemain untuk menampilkan detail tile. Sistem menghitung area klik setiap tile menggunakan boardTileBounds() berdasarkan ukuran papan aktual. Jika tile diklik, sistem menampilkan overlay berisi informasi lengkap seperti nama tile, kode, jenis, harga, nilai gadai, pemilik, status gadai, tabel sewa, biaya bangunan, dan efek festival jika aktif. Overlay ini dapat di-scroll dan ditutup tanpa menghentikan permainan.

5.2.2. Panel pemain interaktif

Panel pemain di sisi kiri dibuat interaktif sehingga setiap baris pemain dapat diklik. Ketika salah satu pemain dipilih, panel properti di bawah papan akan menampilkan daftar properti milik pemain tersebut. Fitur ini memudahkan pemain untuk melihat kondisi lawan tanpa perlu menunggu giliran atau memasukkan perintah tambahan.

5.2.3. Strip kartu properti dengan indikator status

Properti milik pemain ditampilkan dalam bentuk kartu horizontal yang dapat di-scroll. Setiap kartu menampilkan kode properti, warna group, nama properti, level bangunan, status gadai, dan efek festival yang sedang aktif. Properti yang sedang digadaikan diberi badge MRTG, sedangkan properti yang terkena festival diberi badge khusus berisi multiplier dan sisa durasi.

5.2.4. Rendering bidak adaptif

Bidak pemain dirender menggunakan tekstur karakter yang dipilih saat setup. Jika lebih dari satu pemain berada pada tile yang sama, posisi bidak akan disesuaikan secara otomatis agar tidak saling menumpuk. Dua pemain ditampilkan berdampingan, tiga pemain dalam formasi segitiga, dan empat pemain dalam formasi 2×2. Jika tekstur tidak tersedia, sistem menggunakan fallback berupa lingkaran berwarna.

5.2.5. Tampilan elapsed time permainan

Panel ringkasan menampilkan waktu permainan yang telah berjalan dalam format menit dan detik. Fitur ini membantu pemain mengetahui durasi permainan secara langsung dari GUI.

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Task 1. Board, Tiles, dan Property Rules • Model papan dan pengelompokan properti: Board.hpp, Board.cpp, ColorGroup.hpp, dan ColorGroup.cpp. • Seluruh kelas pada folder tiles/, mencakup tile properti dan non-properti. • Struktur papan, pengambilan tile berdasarkan indeks atau kode, kepemilikan properti, perhitungan sewa, status gadai, pembangunan rumah dan hotel, serta aturan color group atau monopoli.	13524068	13524026 13524028 13524034 13524067 13524068
Task 2. Player, Money, Effects, Festival, dan Bank • Model ekonomi dan pemain: Money.hpp, Money.cpp, Player.hpp, dan Player.cpp. • Seluruh komponen pada folder effects/. • FestivalManager.hpp, FestivalManager.cpp, Bank.hpp, dan Bank.cpp. • Saldo pemain, posisi, status, daftar properti, kartu skill, efek aktif, transaksi uang, pembayaran antar pemain, pembayaran dari atau ke bank, efek festival, serta pengaruh efek terhadap pembayaran dan penerimaan sewa.	13524034	13524026 13524028 13524034 13524067 13524068

Task 3. Card System, Deck, dan Seluruh Kartu • Seluruh sistem kartu pada folder cards/. • Deck, kartu Chance, kartu Community Chest, kartu Skill, serta seluruh inheritance antar kelas kartu. • Pembuatan deck, pengambilan kartu, discard pile, pengocokan ulang deck, dan efek kartu yang memengaruhi uang, posisi pemain, properti, perlindungan, diskon, teleportasi, demolisi, lasso, serta aksi khusus lainnya.	13524067	13524026 13524028 13524034 13524067 13524068
Task 4. UI, Input, dan Controller • Seluruh bagian yang berkaitan dengan tampilan, input pemain, dan controller tanpa mengubah logika inti permainan. • Tampilan papan, panel pemain, panel properti, log aksi, prompt input, popup, animasi dadu, dan interaksi klik tile. • Controller pemain: PlayerController.hpp, PlayerController.cpp, HumanController.hpp, HumanController.cpp, ComputerController.hpp, dan ComputerController.cpp. • Cara pemain manusia dan bot memberikan keputusan ke sistem permainan.	13524026 13524028 13524067	13524026 13524028 13524067
Task 5. Game Orchestration, Commands, State, Event/Logging, dan Data Access • Orkestrasi utama permainan: GameEngine.hpp, GameEngine.cpp, GameBuilder.hpp, GameBuilder.cpp, TurnManager.hpp, TurnManager.cpp, AuctionManager.hpp, AuctionManager.cpp, BankruptcyManager.hpp, BankruptcyManager.cpp, EventBus.hpp, EventBus.cpp, CommandRegistry.hpp, dan CommandRegistry.cpp. • Game state dan model pendukung: GameState.hpp, GameState.cpp, GameStateView.hpp, GameStateView.cpp, Dice.hpp, Dice.cpp, GameConfig.hpp, GameConfig.cpp, TransactionLogger.hpp, dan TransactionLogger.cpp. • Komponen models/events, command/, dan data/. • Game loop, pergantian giliran, lelang, kebangkrutan, command permainan, penyimpanan state, pembacaan konfigurasi, event game, logging transaksi, serta akses data untuk save dan load.	13524028	13524028 13524067
BONUS : Com, GUI, Dynamic Board	13524026 13524028 13524034 13524067 13524068	13524028 13524067 13524068

Lampiran

- **Lampiran A. Form Asistensi @ IF2010_TB1_Asistensi_BCC_Signed.pdf**
- **Lampiran B. GUI Design @ Figma**